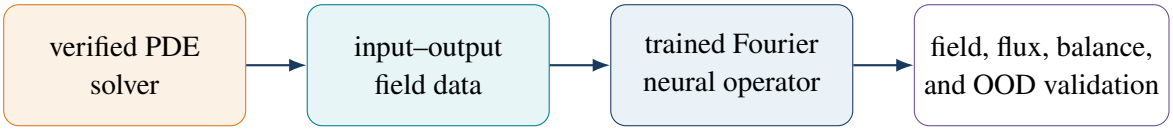
Fourier Neural Operators for Engineering PDEs

Detailed Derivations, Reproduction, and Physical Validation

of the AIDME-2026 Heterogeneous Heat-Conduction Prototype



Central purpose. This note derives every mathematical and computational step behind the prototype results reported in the poster. It distinguishes the physical PDE, the reference discretization, the neural architecture, the statistical metrics, and the additional verification and validation evidence.

Yanwei Wang

Department of Chemical and Materials Engineering, Nazarbayev University

August 2026

Preface

The AIDME-2026 poster presents a compact Fourier neural operator (FNO) prototype for one-dimensional heterogeneous heat conduction. Its purpose is both educational and methodological: it introduces operator learning and demonstrates, using a reproducible numerical experiment, why a small state-field error is not sufficient evidence of physical reliability.

The poster reports four central medians:

$$E_T^{\mathrm{ID}} = 1.6670\%, \qquad E_q^{\mathrm{ID}} = 14.6799\%,$$
$$E_T^{\mathrm{OOD}} = 10.8465\%, \qquad E_q^{\mathrm{OOD}} = 220.6089\%.$$

The first pair refers to unseen smooth conductivity fields drawn from the same family as the training data. The second pair refers to piecewise-constant layered fields deliberately absent from training. The values have been independently recomputed from the archived arrays and reproduced by rerunning the fixed-seed training script.

This note is written for engineering students who want to understand not only *what* the poster reports, but *how* each number arises. It therefore develops five linked levels of reasoning:

1. the continuous heat-conduction problem and its exact integral structure;
2. the conservative flux-form reference discretization;
3. the construction of smooth and layered conductivity ensembles;
4. the actual FNO architecture and training objective used in the prototype;
5. the derivation and interpretation of state, flux, balance, residual, positivity, and out-of-distribution metrics.

Scope of the evidence

The prototype is a fixed-seed, synthetic, one-dimensional computational study. It is not experimental validation, field-scale validation, proof of conservation, or evidence of reliable subsurface prediction. The note repeatedly separates computational reproducibility from physical validation.

How to use this note

Students encountering FNOs for the first time should read Chapters 1–9 in sequence. Readers already familiar with Fourier analysis and neural operators may begin with Chapter 3, which derives the continuous solution, and then proceed to Chapters 4, 8, 10, and 11 for the reference solver, actual network, metrics, and audited results.

Each chapter contains one or more of the following:

- **derivation boxes**, showing algebraic steps that are compressed in the poster;
- **evidence boxes**, reporting quantities read directly from the archived data and audit;
- **engineering interpretation boxes**, connecting mathematical errors to physical meaning;
- **checkpoints**, suitable for tutorials or independent study;
- explicit qualifications that prevent a numerical demonstration from being overstated.

Contents

Preface i

How to use this note ii

I From the Poster to a Traceable Scientific Claim 1

1 What the Poster Claims and What Must Be Derived 2
1.1 The poster as a compressed evidence chain . . . 2
1.2 Verification and validation are different questions . . . 2
1.3 Claim-to-evidence map . . . 2

2 From a Parametric PDE to a Solution Operator 5
2.1 Fields rather than scalar outputs . . . 5
2.2 Admissible conductivity fields . . . 5
2.3 Weak formulation . . . 5
2.4 Why this is a useful educational operator . . . 6

II Physical and Numerical Derivations 7

3 Exact Continuous Structure of the Heat-Conduction Problem 8
3.1 Introduce the conductive heat flux . . . 8
3.2 Determine the integration constant . . . 8
3.3 Resistance-coordinate form . . . 9
3.4 Constant-conductivity special case . . . 10
3.5 Positivity and maximum-principle behavior . . . 10
3.6 Nonlocal dependence on conductivity . . . 10

4 Flux-Form Discretization and the Reference Solver 11
4.1 Grid and unknowns . . . 11
4.2 Face fluxes . . . 11
4.3 Why the harmonic mean is used . . . 11
4.4 Tridiagonal equation . . . 12
4.5 Positive definiteness and uniqueness . . . 12
4.6 Why the constant-conductivity solution is exact at nodes . . . 12
4.7 Discrete global balance . . . 13
4.8 Thomas algorithm . . . 13

5 Verification of the Reference Solver 14
5.1 Independent exact reference by quadrature . . . 14
5.2 Observed convergence . . . 14
5.3 Qualification for discontinuous layered fields . . . 14

III Dataset, Fourier Operator, and Training Derivations 16

6 Construction of the Training and Test Distributions 17
6.1 Dataset sizes and deterministic ordering . . . 17
6.2 Smooth log-Fourier family . . . 17
6.3 Layered OOD family . . . 17
6.4 Why this is a structured OOD test . . . 19
6.5 No exact train–test duplicates . . . 19

7 From an Integral Operator to a Fourier Neural Operator 20
7.1 A nonlocal neural-operator layer . . . 20
7.2 Convolution theorem . . . 20
7.3 Multiple latent channels . . . 20
7.4 Discrete real FFT . . . 21
7.5 Pointwise path and nonlinearity . . . 21
7.6 Global receptive field . . . 21
7.7 Computational cost . . . 22

8 The Actual Prototype Architecture 23
8.1 Input normalization . . . 23
8.2 Lifting, residual blocks, and projection . . . 23
8.3 Why the boundary error is exactly zero . . . 23
8.4 Spectral tensor shape . . . 24
8.5 Detailed parameter count . . . 24

9 Training Objective and Optimization Protocol 26
9.1 Supervised field loss . . . 26
9.2 First-difference penalty . . . 26
9.3 Optimizer and scheduler . . . 26
9.4 Determinism and its limits . . . 27

IV Metrics, Results, and Physical Interpretation 28

10 Derivation of the Evaluation Metrics 29
10.1 Temperature-field relative error . . . 29
10.2 Heat-flux relative error . . . 29
10.3 Why differentiation magnifies error . . . 29
10.4 Discrete balance error . . . 30
10.5 Local PDE residual . . . 30
10.6 Boundary and positivity diagnostics . . . 30
10.7 Bootstrap interval for the median . . . 31

11 Audited Numerical Results 32
11.1 Four principal medians . . . 32
11.2 Full state-error distributions . . . 33
11.3 Full flux-error distributions . . . 33
11.4 Physical diagnostics . . . 33
11.5 What the training history does and does not show . . . 35

12 Representative Cases and Local Error Structure 36

12.1 How the representative cases are selected . . . 36
12.2 Why the ID temperature profile can look excellent . . . 36
12.3 Why layered fields are difficult . . . 36
12.4 A spectral interpretation with an important caveat . . . 38

13 Independent Reproduction and Audit Trail 39
13.1 Archived checkpoint prediction check . . . 39
13.2 Independent metric recomputation . . . 39
13.3 Independent retraining . . . 39
13.4 File integrity . . . 39
13.5 Reproducibility hierarchy . . . 39

V Scientific Interpretation and Next Steps 41

14 What the Results Support – and What They Do Not 42
14.1 Supported statement . . . 42
14.2 Statements not supported by the experiment . . . 42
14.3 Why median temperature error alone is insufficient . . . 42
14.4 Model card for the prototype . . . 43

15 How to Improve the Prototype Scientifically 44
15.1 Multi-seed evaluation . . . 44
15.2 Comparable baselines . . . 44
15.3 Flux-aware and residual-aware loss . . . 44
15.4 Conservative output design . . . 44
15.5 Mixed smooth-layered training . . . 45
15.6 True resolution-transfer experiment . . . 45

16 Pathway to Porous-Media and Energy Applications 46
16.1 Single-phase Darcy analogue . . . 46
16.2 Why the heat prototype is only a first step . . . 46
16.3 Global-local conservative operator . . . 46
16.4 Validation ladder for future energy models . . . 46

VI Student Exercises and Reproducibility Materials 48

17 Guided Exercises 49
17.1 Analytical exercises . . . 49
17.2 Numerical exercises . . . 49
17.3 Machine-learning exercises . . . 49
17.4 Discussion questions . . . 50

A Key Code Excerpts 51
A.1 Reference solver . . . 51
A.2 Conductivity generators . . . 51
A.3 Spectral layer and residual block . . . 52
A.4 Training and metric calculation . . . 54

B Complete Audited Statistics 56

B.1 Additional physical diagnostics 58

C Selected Solutions to Exercises **59**
C.1 Bound on the integration constant 59
C.2 Location of the temperature maximum 59
C.3 Parameter count with eight modes 59

D Reproduction Guide **60**

E Notation **61**

List of Figures

1.1 The AIDME-2026 poster whose equations and numerical results are derived in this note. 3

3.1 Continuous solution construction for a smooth conductivity. The flux is linear, while the conductivity controls the temperature gradient needed to carry that flux. The $N = 64$ reference solution lies almost exactly on the independent quadrature solution. 9

4.1 Flux-form stencil. The source is balanced by the difference between outgoing and incoming face fluxes. 11

5.1 Independent verification of the reference solver. The constant-conductivity case is exact at nodes, and the two smooth variable-conductivity cases converge at approximately second order. 15

6.1 Examples of the smooth training family and the held-out layered family. Both are positive, but their spatial regularity is qualitatively different. 18

6.2 Mean spectral energy of the archived conductivity families. The smooth family is concentrated in low modes, whereas discontinuities give layered fields a broader spectrum. The shaded region marks the 16 retained real-FFT modes in each spectral layer. 19

7.1 Fourier-space mixing. The learned multiplier acts mode by mode, while each retained mode has global spatial support. 21

8.1 Exact architecture implemented in the supplied training script. The boundary condition is hard-coded after projection. 24

8.2 Breakdown of tensor elements and real scalar degrees of freedom. Complex spectral weights dominate the model capacity. 25

9.1 Archived fixed-seed training history and the cosine learning-rate schedule. The plotted training objective and validation relative error are different quantities and should not be compared as if they had identical definitions. 27

10.1 Case-by-case relationship between state and flux errors. Flux error is consistently larger and becomes extreme for layered OOD fields. 30

11.1 Audited medians with bootstrap intervals for the fixed trained model and fixed test families. 32

11.2 Error distributions over the two 250-case test families. The logarithmic scale is essential because OOD flux errors are very large. 34

11.3 Balance and positivity diagnostics. These reveal physical failures not visible in the median temperature error. 34

12.1 Detailed diagnostics for the median-temperature representative cases. The ID temperature curves nearly overlap, but its flux and residual are visibly less accurate. The layered case shows severe flux and residual errors, including a negative temperature region. 37

List of Tables

1.1 Main poster statements and the derivations required to support them. 4

5.1 Relative L^2 errors for the two smooth verification cases. 14

8.1 Parameter-count derivation for the actual prototype. 24

14.1 Compact model card for the audited prototype. 43

B.1 Audited relative-error statistics. All values are percentages except where noted. 57

B.2 Physical diagnostics of the direct temperature predictor. 58

Part I

From the Poster to a Traceable Scientific Claim

Chapter 1

What the Poster Claims and What Must Be Derived

1.1 The poster as a compressed evidence chain

A scientific poster necessarily compresses a long workflow. In this case, the displayed chain is

$$\text{physical PDE} \longrightarrow \text{verified numerical data} \longrightarrow \text{trained FNO} \longrightarrow \text{field and physical metrics.} \tag{1.1}$$

The poster page is reproduced in figure 1.1. It contains the operator formulation, the Fourier layer, the prototype heat equation, the training history, the four principal error medians, representative cases, and the stated limitations.

The poster's central operator is

$$\mathcal{G}_\theta : [\log k(x), x] \longmapsto T_\theta(x), \tag{1.2}$$

where the reference field satisfies

$$-\frac{\mathrm{d}}{\mathrm{d}x}\left[k(x)\frac{\mathrm{d}T}{\mathrm{d}x}\right] = 1, \qquad 0 < x < 1, \qquad T(0) = T(1) = 0. \tag{1.3}$$

1.2 Verification and validation are different questions

In computational engineering, *verification* and *validation* answer different questions.

Verification

Verification asks whether the equations have been implemented and solved correctly. In this note it includes checking the finite-volume solver against exact continuous solutions, recomputing all archived metrics, loading the checkpoint into the supplied model class, and rerunning the fixed-seed experiment.

Validation

Validation asks whether the model is adequate for its intended physical use. In this prototype it includes heat-flux accuracy, discrete balance, residuals, positivity, and structured out-of-distribution behavior. Experimental and field-scale validation are outside the scope of the dataset.

A model can pass a reproducibility check and still fail a physical adequacy check. This distinction is one of the most important lessons of the poster.

1.3 Claim-to-evidence map



Figure 1.1: The AIDME-2026 poster whose equations and numerical results are derived in this note.

Table 1.1: Main poster statements and the derivations required to support them.

Poster statement	**Required derivation or check**	**Where developed**
The PDE defines a solution operator	Establish well-posed field-to-field mapping and identify all varying inputs	Chapters 2–3
Reference data are generated by a finite-volume solver	Derive face fluxes, harmonic means, tridiagonal system, and balance	Chapter 4
The reference solver is verified	Compare with an independent continuous quadrature solution and perform grid refinement	Chapter 5
The model is a four-block FNO with width 32 and 16 modes	Derive spectral convolution, residual blocks, and exact tensor shapes	Chapters 7–8
The model has 72,033 parameters	Reconcile complex tensor elements with real scalar degrees of freedom	Section 8.5
ID temperature median is 1.6670%	Recompute relative L^2 errors over 250 unseen smooth fields	Chapter 11
ID heat-flux median is 14.6799%	Derive harmonic-face flux and differentiate predicted fields	Chapters 10–11
Layered OOD errors are larger	Define the held-out family and show the distribution shift is structural	Chapters 6 and 11
Physical quantities need separate validation	Derive balance, residual, boundary, and positivity diagnostics	Chapter 10
The workflow is reproducible	Check fixed seed, unique datasets, model checkpoint, hashes, and rerun	Chapter 13

Chapter summary

The poster's numerical values are endpoints of a chain of physical, numerical, statistical, and machine-learning choices. A trustworthy interpretation requires every link in that chain to be explicit.

Chapter 2

From a Parametric PDE to a Solution Operator

2.1 Fields rather than scalar outputs

An ordinary regression model may predict one scalar response from a finite list of inputs. The prototype instead receives an entire conductivity field and returns an entire temperature field. Mathematically, the physical solution process is

$$\mathcal{G}^{\dagger} : k(\cdot) \mapsto T(\cdot). \tag{2.1}$$

The dagger identifies the trusted reference operator, while $\mathcal{G}_\theta$ denotes its learned approximation.

For the fixed source and fixed zero-temperature boundaries of equation (1.3), the only physical field that varies between cases is $k(x)$. The coordinate x is included as a network input because the Fourier convolution itself is translation equivariant, while the boundary-value problem is tied to absolute position.

2.2 Admissible conductivity fields

The continuous problem requires

$$0 < k_{\min} \le k(x) \le k_{\max} < \infty. \tag{2.2}$$

This positivity condition ensures uniform ellipticity and gives a unique weak solution for the Dirichlet problem. It is also physically natural because thermal conductivity must be positive.

Engineering interpretation

The network input is $\log k$, not k. Taking the logarithm converts positivity into an unconstrained real-valued field and makes multiplicative conductivity contrasts easier to represent numerically.

2.3 Weak formulation

Multiply equation (1.3) by a test function $v \in H_0^1(0,1)$ and integrate:

$$-\int_0^1 \frac{\mathrm{d}}{\mathrm{d}x}\left(kT'\right) v \,\mathrm{d}x = \int_0^1 v \,\mathrm{d}x. \tag{2.3}$$

Integration by parts, together with $v(0) = v(1) = 0$, gives

$$\int_0^1 k(x)T'(x)v'(x)\,\mathrm{d}x = \int_0^1 v(x)\,\mathrm{d}x. \tag{2.4}$$

The bilinear form

$$a(T, v) = \int_0^1 kT'v' \,\mathrm{d}x \tag{2.5}$$

is coercive under equation (2.2). The weak formulation therefore defines a unique solution operator. This field-to-field viewpoint is the mathematical basis of neural-operator learning [1].

2.4 Why this is a useful educational operator

The problem is deliberately simple enough to verify independently, but rich enough to expose central operator-learning issues:

- the input and output are full fields;
- the response is nonlocal because conductivity anywhere affects the integration constant and therefore the solution everywhere;
- the reference solver is cheap and verifiable;
- heat flux depends on a derivative and is more demanding than temperature;
- smooth and layered conductivity families create a transparent interpolation-versus-extrapolation test.

Check your understanding

Suppose the source amplitude, left boundary temperature, and right boundary temperature vary between samples. Write the complete operator input. Explain why omitting any one of these varying quantities makes the supervised learning problem ambiguous.

Part II

Physical and Numerical Derivations

Chapter 3

Exact Continuous Structure of the Heat-Conduction Problem

3.1 Introduce the conductive heat flux

Define the one-dimensional conductive flux

$$q_h(x) = -k(x)\frac{\mathrm{d}T}{\mathrm{d}x}. \tag{3.1}$$

Substituting this definition into equation (1.3) gives

$$\frac{\mathrm{d}q_h}{\mathrm{d}x} = 1. \tag{3.2}$$

The distributed source therefore makes the heat flux vary linearly with position:

$$q_h(x) = x + C, \tag{3.3}$$

where C is an integration constant determined by the two boundary temperatures.

3.2 Determine the integration constant

From equations (3.1) and (3.3),

$$T'(x) = -\frac{x + C}{k(x)}. \tag{3.4}$$

Integrating from 0 to x and using $T(0) = 0$ yields

$$T(x) = -\int_0^x \frac{s + C}{k(s)}\,\mathrm{d}s. \tag{3.5}$$

The second boundary condition $T(1) = 0$ requires

$$0 = -\int_0^1 \frac{s + C}{k(s)}\,\mathrm{d}s. \tag{3.6}$$

Therefore

$$C\int_0^1 \frac{1}{k(s)}\,\mathrm{d}s = -\int_0^1 \frac{s}{k(s)}\,\mathrm{d}s, \tag{3.7}$$

and

$$\boxed{C = -\frac{\displaystyle\int_0^1 s/k(s)\,\mathrm{d}s}{\displaystyle\int_0^1 1/k(s)\,\mathrm{d}s}.} \tag{3.8}$$

3.3 Resistance-coordinate form

Define cumulative thermal-resistance integrals

$$R(x) = \int_0^x \frac{1}{k(s)}\,\mathrm{d}s, \qquad A(x) = \int_0^x \frac{s}{k(s)}\,\mathrm{d}s. \tag{3.9}$$

Then equations (3.5) and (3.8) give

$$\boxed{T(x) = \frac{A(1)}{R(1)}R(x) - A(x).} \tag{3.10}$$

Equation (3.10) is an exact quadrature representation for any positive integrable conductivity field.

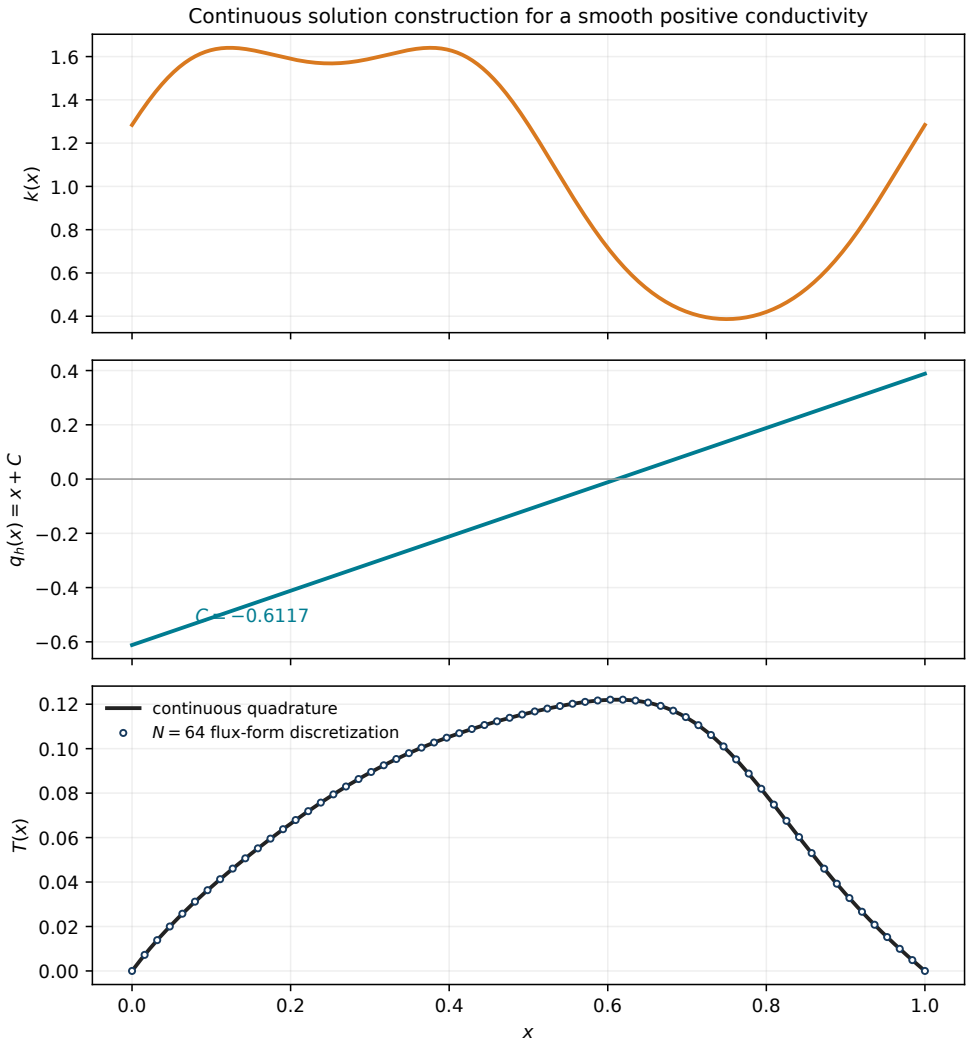


Figure 3.1: Continuous solution construction for a smooth conductivity. The flux is linear, while the conductivity controls the temperature gradient needed to carry that flux. The $N = 64$ reference solution lies almost exactly on the independent quadrature solution.

3.4 Constant-conductivity special case

If $k(x) = k_0$, then

$$C = -\frac{\int_0^1 s\,\mathrm{d}s}{\int_0^1 1\,\mathrm{d}s} = -\frac{1}{2}. \tag{3.11}$$

Hence

$$T'(x) = -\frac{x - 1/2}{k_0}, \tag{3.12}$$

and

$$\boxed{T(x) = \frac{x(1-x)}{2k_0}}. \tag{3.13}$$

The maximum occurs at $x = 1/2$ and equals $1/(8k_0)$.

3.5 Positivity and maximum-principle behavior

For positive k and positive source, the exact solution is nonnegative. A simple smooth-function argument is instructive. If T had a negative interior minimum at x_*, then $T'(x_*) = 0$ and $T''(x_*) \geq 0$. At that point,

$$-(kT')' = -kT'' - k'T' = -kT'' \leq 0, \tag{3.14}$$

which contradicts the source value $1 > 0$. Thus

$$T(x) \geq 0. \tag{3.15}$$

This physical property later becomes a validation test for the FNO.

3.6 Nonlocal dependence on conductivity

Equation (3.8) shows that changing k in one region changes the global constant C. Consequently, it changes $T'(x)$ and $T(x)$ throughout the domain. The problem is therefore nonlocal even though the governing equation is written locally.

Key idea

The global receptive field of a Fourier layer is relevant because the true solution operator is globally coupled: local conductivity changes modify both local resistance and a domain-wide flux constant.

Check your understanding

Derive the exact solution when the source is a general function $f(x)$ rather than 1. Introduce $F(x) = \int_0^x f(s)\,\mathrm{d}s$, derive $q_h(x) = F(x) + C$, and determine C from the two boundary conditions.

Chapter 4

Flux-Form Discretization and the Reference Solver

4.1 Grid and unknowns

The reference discretization follows a standard conservative flux-form construction used in heat-transfer and finite-volume analysis [2, 3]. The prototype uses $N = 64$ equally spaced nodal points

$$x_j = j\Delta x, \qquad j = 0, 1, \ldots, N-1, \qquad \Delta x = \frac{1}{N-1} = \frac{1}{63}. \tag{4.1}$$

The boundary values are fixed:

$$T_0 = T_{N-1} = 0. \tag{4.2}$$

The unknown vector contains the $N - 2 = 62$ internal temperatures.

4.2 Face fluxes

At face $j + 1/2$, the discrete conductive flux is

$$q_{j+1/2} = -k_{j+1/2}\frac{T_{j+1} - T_j}{\Delta x}. \tag{4.3}$$

The nodal balance is

$$\frac{q_{j+1/2} - q_{j-1/2}}{\Delta x} = 1, \qquad j = 1, \ldots, N-2. \tag{4.4}$$

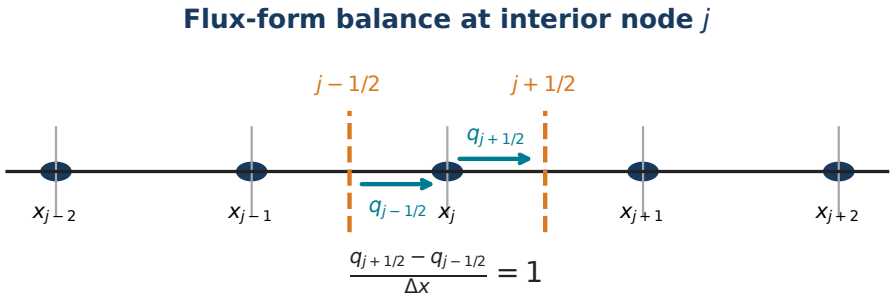


Figure 4.1: Flux-form stencil. The source is balanced by the difference between outgoing and incoming face fluxes.

4.3 Why the harmonic mean is used

Suppose the interval between nodes j and $j + 1$ is treated as two half-length materials in series. The thermal resistance is

$$\mathcal{R}_{j+1/2} = \frac{\Delta x/2}{k_j} + \frac{\Delta x/2}{k_{j+1}}. \tag{4.5}$$

Define an equivalent face conductivity $k_{j+1/2}$ by

$$\mathcal{R}_{j+1/2} = \frac{\Delta x}{k_{j+1/2}}. \tag{4.6}$$

Equating the two expressions gives

$$\frac{1}{k_{j+1/2}} = \frac{1}{2}\left(\frac{1}{k_j} + \frac{1}{k_{j+1}}\right), \tag{4.7}$$

$$\boxed{k_{j+1/2}} = \boxed{\frac{2k_j k_{j+1}}{k_j + k_{j+1}}}. \tag{4.8}$$

The harmonic mean is especially important across strong conductivity jumps because heat transfer is controlled by resistance rather than arithmetic averaging.

4.4 Tridiagonal equation

Substitute equation (4.3) into equation (4.4):

$$-\frac{1}{\Delta x}\left[-k_{j+1/2}\frac{T_{j+1} - T_j}{\Delta x} + k_{j-1/2}\frac{T_j - T_{j-1}}{\Delta x}\right] = 1. \tag{4.9}$$

After collecting coefficients,

$$-\frac{k_{j-1/2}}{\Delta x^2}T_{j-1} + \frac{k_{j-1/2} + k_{j+1/2}}{\Delta x^2}T_j - \frac{k_{j+1/2}}{\Delta x^2}T_{j+1} = 1. \tag{4.10}$$

The system is tridiagonal and is solved with the Thomas algorithm.

4.5 Positive definiteness and uniqueness

For an internal vector z extended by zero boundary values, the quadratic form satisfies

$$z^T A z = \frac{1}{\Delta x^2}\sum_{j=0}^{N-2} k_{j+1/2}(z_{j+1} - z_j)^2 > 0 \tag{4.11}$$

for every nonzero z when $k_{j+1/2} > 0$. Hence A is symmetric positive definite. The discrete solution is unique.

The matrix is also an M-matrix: diagonal entries are positive, off-diagonal entries are nonpositive, and diagonal dominance follows from the flux form. This structure helps preserve nonnegativity for a nonnegative source.

4.6 Why the constant-conductivity solution is exact at nodes

For $k = k_0$, the exact solution equation (3.13) is quadratic. The centered second-difference operator differentiates a quadratic exactly:

$$\frac{T_{j+1} - 2T_j + T_{j-1}}{\Delta x^2} = -\frac{1}{k_0}. \tag{4.12}$$

Therefore the discrete equation is satisfied exactly up to floating-point roundoff. This explains why the constant-k verification errors are of order 10^{-15} rather than merely second order.

4.7 Discrete global balance

Summing equation (4.4) over $j = 1, \ldots, N-2$ gives telescoping cancellation:

$$\sum_{j=1}^{N-2} \left(q_{j+1/2} - q_{j-1/2}\right) = q_{N-3/2} - q_{1/2}. \tag{4.13}$$

The right-hand side is

$$\sum_{j=1}^{N-2} \Delta x = (N-2)\Delta x = \frac{62}{63}. \tag{4.14}$$

Thus the discrete reference fields satisfy

$$q_{N-3/2} - q_{1/2} = \frac{62}{63} \tag{4.15}$$

up to roundoff.

Why the discrete source total is $62/63$, not 1

The code writes source equations only at the 62 internal nodes and assigns each equation a full width Δx. The resulting algebraic diagnostic is exactly consistent with the implemented stencil. A different finite-volume layout using boundary half-control-volumes could integrate the source over the full unit interval. The audit intentionally checks the balance of the implemented discrete system.

4.8 Thomas algorithm

For a tridiagonal system

$$a_j T_{j-1} + b_j T_j + c_j T_{j+1} = d_j, \tag{4.16}$$

forward elimination replaces each row by

$$m_j = \frac{a_j}{b_{j-1}}, \tag{4.17}$$

$$b_j \leftarrow b_j - m_j c_{j-1}, \tag{4.18}$$

$$d_j \leftarrow d_j - m_j d_{j-1}. \tag{4.19}$$

Back substitution then gives

$$T_{N-2} = \frac{d_{N-2}}{b_{N-2}}, \tag{4.20}$$

$$T_j = \frac{d_j - c_j T_{j+1}}{b_j}. \tag{4.21}$$

This direct algorithm has $O(N)$ cost and is numerically appropriate for the positive tridiagonal system.

Chapter summary

The reference solver is a conservative flux-form discretization with harmonic face conductivity. Its tridiagonal structure, positive definiteness, exact constant-k behavior, and independently verified convergence make it a suitable data generator for the educational prototype.

Chapter 5

Verification of the Reference Solver

5.1 Independent exact reference by quadrature

The verification script does not compare the solver with itself. It uses equations (3.5) and (3.8) and high-accuracy numerical quadrature to construct an independent continuous reference. Three cases are checked:

1. $k(x) = 1$;
2. $k(x) = \exp[0.7\sin(2\pi x) + 0.25\cos(4\pi x)]$;
3. $k(x) = \exp[-0.2 + 0.8\sin(2\pi x + 0.3) + 0.18\cos(8\pi x)]$.

5.2 Observed convergence

For a sequence of grids $N = 16, 32, 64, 128, 256$, define

$$E_N = \frac{\|T_N - T_{\text{exact}}\|_2}{\|T_{\text{exact}}\|_2}. \tag{5.1}$$

The observed order between two refinements is

$$p_N = \frac{\log(E_N/E_{2N})}{\log 2}. \tag{5.2}$$

For the smooth cases, p_N approaches 2, consistent with a second-order flux-form discretization.

Table 5.1: Relative L^2 errors for the two smooth verification cases.

N	smooth case 1	smooth case 2	p case 1	p case 2
16	0.9345%	1.1796%	–	–
32	0.2198%	0.2783%	2.09	2.08
64	0.0533%	0.0675%	2.04	2.04
128	0.0131%	0.0166%	2.02	2.02
256	0.00325%	0.00412%	2.01	2.01

At the $N = 64$ resolution used to generate the machine-learning data, the solver error is approximately 0.05–0.07% for these smooth analytical tests, substantially below the FNO's median ID temperature error of 1.6670%.

5.3 Qualification for discontinuous layered fields

The layered test fields are represented as nodal piecewise-constant arrays. A conductivity jump is treated by a harmonic face value. The archived target is therefore the solution of the stated $N = 64$ discrete problem. Continuous-PDE convergence for discontinuous materials additionally depends on interface placement and grid alignment.

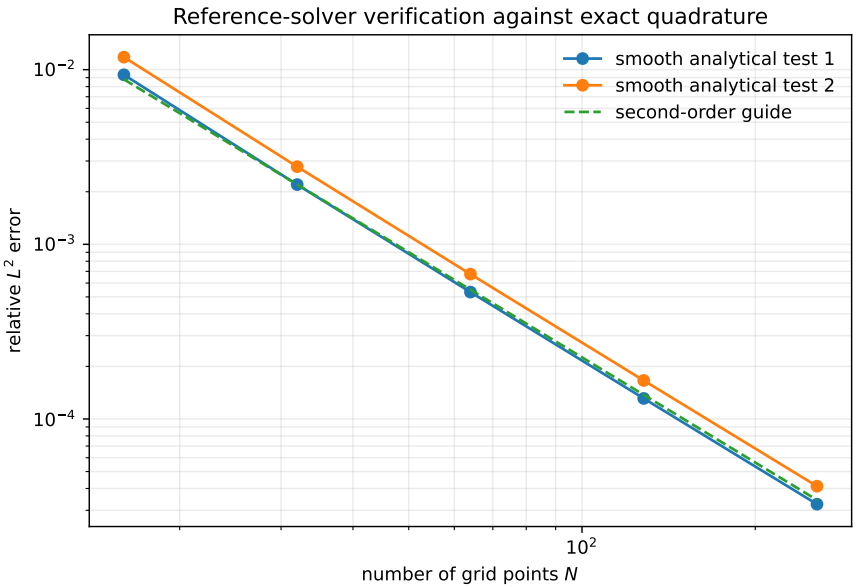


Figure 5.1: Independent verification of the reference solver. The constant-conductivity case is exact at nodes, and the two smooth variable-conductivity cases converge at approximately second order.

Important qualification

The OOD results validate the FNO against a reproducible discrete reference, not against a laboratory conductor. This distinction must be preserved when the prototype is presented to students.

Part III

Dataset, Fourier Operator, and Training Derivations

Chapter 6

Construction of the Training and Test Distributions

6.1 Dataset sizes and deterministic ordering

The fixed seed is 20260805. NumPy, Python's random module, and PyTorch are seeded before any data or model parameters are generated. The calls are made sequentially, producing:

$$\text{1000 training,} \qquad \text{150 validation,} \qquad \text{250 ID test,} \qquad \text{250 layered OOD test cases.} \tag{6.1}$$

Because one pseudorandom sequence is consumed in a fixed order, reproducing the same software path reproduces the arrays exactly.

6.2 Smooth log-Fourier family

For each smooth case, the code samples

$$\mu_k \sim \mathcal{U}(-0.3, 0.3), \tag{6.2}$$

$$A \sim \mathcal{U}(0.4, 0.8), \tag{6.3}$$

$$\xi_m, \zeta_m \sim \mathcal{N}(0, 1), \qquad m = 1, \dots, 5, \tag{6.4}$$

and constructs

$$\ell(x) = \mu_k + \sum_{m=1}^{5} \frac{A}{m^{1.7}} \left[\xi_m \sin(2\pi m x) + \zeta_m \cos(2\pi m x)\right]. \tag{6.5}$$

If the sampled standard deviation of ℓ exceeds 0.9, deviations from the mean are rescaled so that the standard deviation is 0.9. The conductivity is

$$k(x) = \exp[\ell(x)] > 0. \tag{6.6}$$

The power-law factor $m^{-1.7}$ suppresses high modes and makes the fields spatially smooth.

6.3 Layered OOD family

For each layered case:

1. the number of layers is sampled uniformly from $\{3, 4, 5, 6\}$;
2. cut indices are sampled without replacement away from the two boundaries;
3. each layer value is independently sampled log-uniformly from $[0.25, 4]$;
4. the field is piecewise constant over the selected index intervals.

The layered family is never used in training or validation.

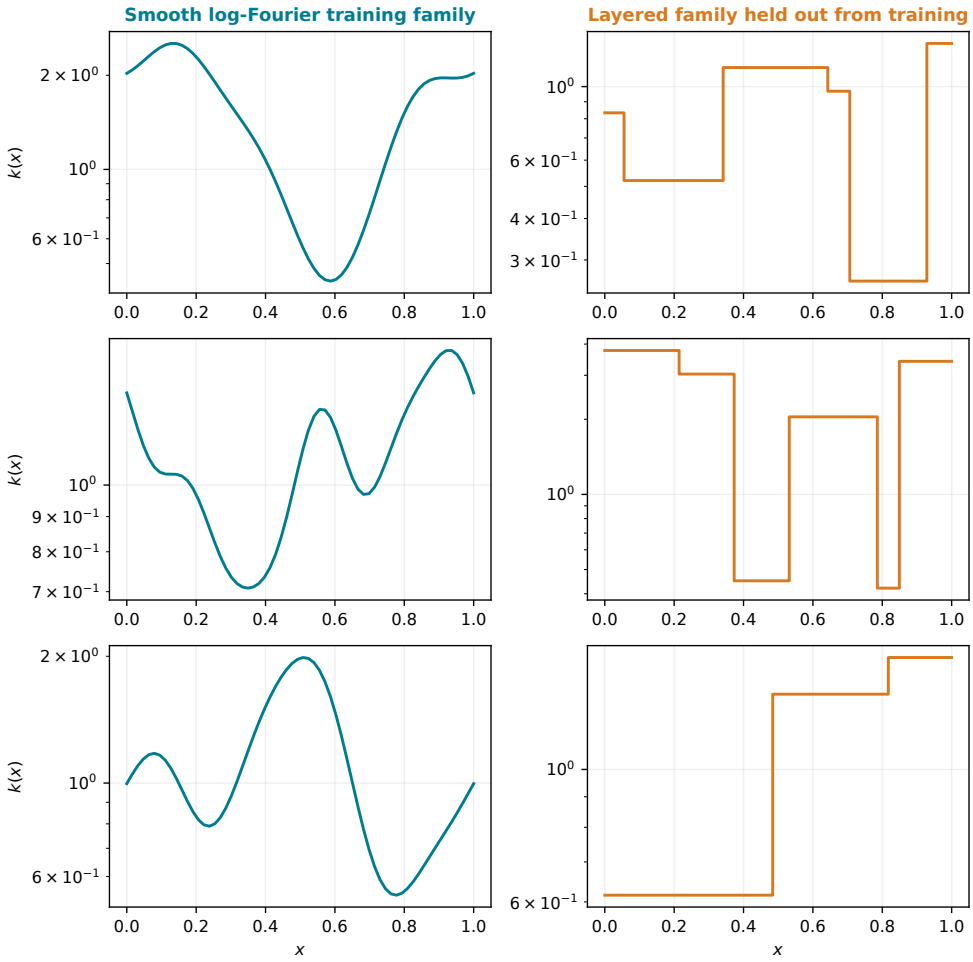


Figure 6.1: Examples of the smooth training family and the held-out layered family. Both are positive, but their spatial regularity is qualitatively different.

6.4 Why this is a structured OOD test

The global conductivity ranges are

$$\text{training:} \quad 0.131 \le k \le 8.345, \tag{6.7}$$

$$\text{ID test:} \quad 0.164 \le k \le 6.155, \tag{6.8}$$

$$\text{layered OOD:} \quad 0.250 \le k \le 3.999. \tag{6.9}$$

The layered values lie inside the global training extrema. Therefore the principal shift is not an unseen scalar range. It is an unseen spatial structure: smooth low-frequency fields versus discontinuous layered fields.

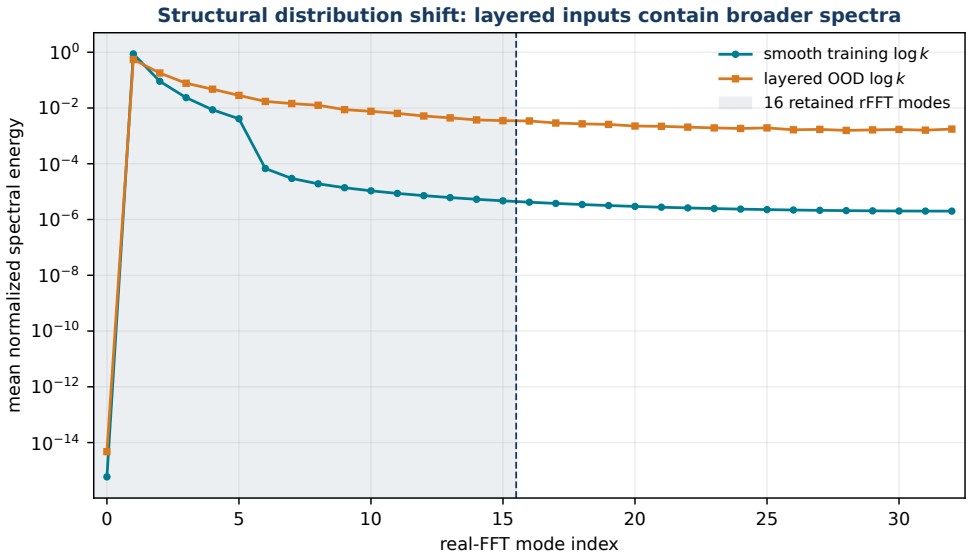


Figure 6.2: Mean spectral energy of the archived conductivity families. The smooth family is concentrated in low modes, whereas discontinuities give layered fields a broader spectrum. The shaded region marks the 16 retained real-FFT modes in each spectral layer.

For the archived arrays, more than 99.99% of the median smooth $\log k$ spectral energy lies in modes 0–15, while the median layered fraction is approximately 96.6%. The remaining few percent can still be important because discontinuities and interface locations are encoded by high-frequency content.

6.5 No exact train–test duplicates

Byte-level hashes show:

$$\text{unique training} = 1000, \quad \text{unique ID} = 250, \quad \text{unique OOD} = 250, \tag{6.10}$$

with zero exact overlap between training and either test family. This check rules out one simple form of data leakage.

Engineering interpretation

Uniqueness is necessary but not sufficient for independence. Random fields generated from the same stochastic family remain statistically related. That is appropriate for an interpolation test, while the layered family supplies a stronger structural shift.

Chapter 7

From an Integral Operator to a Fourier Neural Operator

7.1 A nonlocal neural-operator layer

A generic neural-operator layer can be written as [1]

$$v_{\ell+1}(x) = \sigma\left[W_\ell v_\ell(x) + \int_0^1 \kappa_\ell(x,y)v_\ell(y)\,\mathrm{d}y + b_\ell\right]. \tag{7.1}$$

The integral term allows information at every point y to affect the output at x.

If the kernel depends only on displacement,

$$\kappa_\ell(x,y) = \kappa_\ell(x-y), \tag{7.2}$$

then the integral is a convolution:

$$(\kappa_\ell * v_\ell)(x) = \int_0^1 \kappa_\ell(x-y)v_\ell(y)\,\mathrm{d}y. \tag{7.3}$$

7.2 Convolution theorem

The Fourier transform converts convolution into multiplication:

$$\mathcal{F}[\kappa * v](k) = \widehat{\kappa}(k)\widehat{v}(k). \tag{7.4}$$

Thus

$$(\kappa * v)(x) = \mathcal{F}^{-1}\left[\widehat{\kappa}(k)\widehat{v}(k)\right](x). \tag{7.5}$$

An FNO replaces the known multiplier $\widehat{\kappa}(k)$ with a trainable complex matrix $R_\ell(k)$, following the spectral operator architecture introduced by Li et al. [4].

7.3 Multiple latent channels

Let v_ℓ have C_{in} channels. For a retained mode k, the output coefficient in channel c_o is

$$\widehat{z}_{\ell,c_o}(k) = \sum_{c_i=1}^{C_{\text{in}}} R_{\ell,c_o,c_i}(k)\widehat{v}_{\ell,c_i}(k). \tag{7.6}$$

Therefore

$$R_\ell(k) \in \mathbb{C}^{C_{\text{out}}\times C_{\text{in}}}. \tag{7.7}$$

The complex matrix changes amplitude, phase, and channel composition for each retained mode.

7.4 Discrete real FFT

For $N = 64$ real-valued grid samples, `rfft` stores

$$N/2 + 1 = 33 \tag{7.8}$$

nonredundant complex coefficients. The prototype retains the first 16 coefficients. Mode truncation is a spectral approximation: it reduces cost and capacity while limiting the directly represented short wavelengths [5]. If $\widehat{v}$ denotes the real-FFT output, the spectral branch is

$$\widehat{z}(:, :, 0{:}16) = R(:, :, 0{:}16)\widehat{v}(:, :, 0{:}16), \tag{7.9}$$

$$\widehat{z}(:, :, 16{:}33) = 0, \tag{7.10}$$

$$z = \mathrm{irfft}(\widehat{z}). \tag{7.11}$$

7.5 Pointwise path and nonlinearity

The spectral branch is added to a 1×1 pointwise channel-mixing convolution:

$$s_\ell(x) = \mathcal{K}_\ell v_\ell(x) + W_\ell v_\ell(x). \tag{7.12}$$

The pointwise path preserves full-grid local information and does not undergo mode truncation. A GELU activation makes the block nonlinear.

7.6 Global receptive field

Every Fourier coefficient is computed from all spatial samples. Changing a retained coefficient alters the inverse transform throughout the domain. One spectral layer therefore has domain-wide reach, unlike a small physical-space convolution whose immediate receptive field is local.

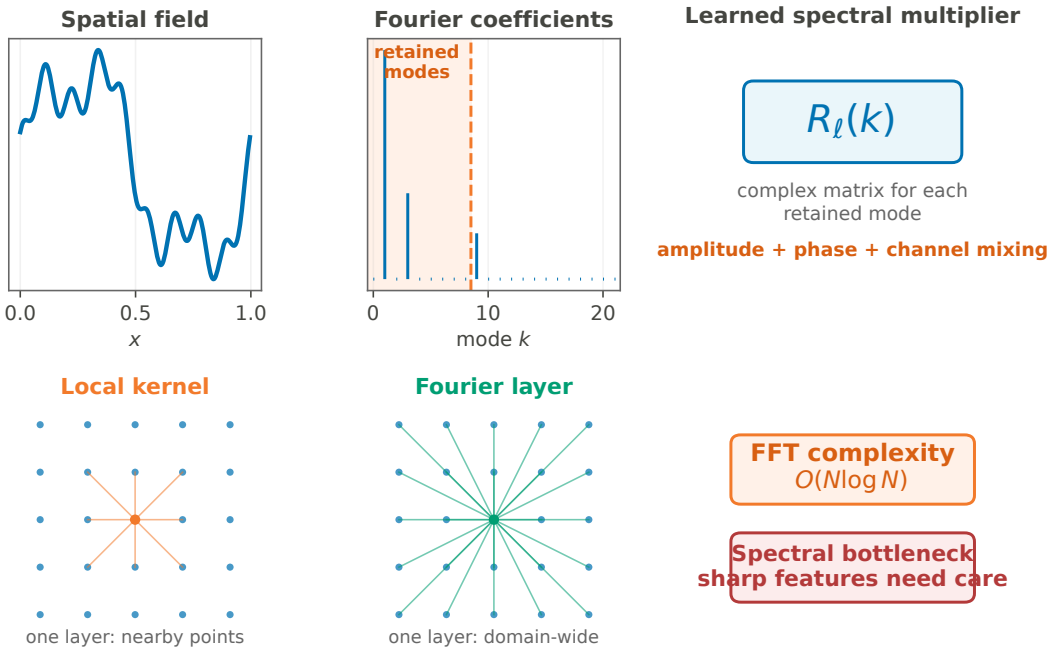


Figure 7.1: Fourier-space mixing. The learned multiplier acts mode by mode, while each retained mode has global spatial support.

7.7 Computational cost

With N grid points, latent width C, and M retained modes, a simplified layer cost is

$$O(CN \log N) + O(C^2 M). \tag{7.13}$$

The first term represents forward and inverse FFTs; the second represents complex channel mixing on retained modes. This is an architectural cost estimate, not a measured speedup for the prototype. Universal-approximation and equation-specific error analyses establish representational results under stated assumptions, but they do not guarantee successful training or extrapolation for this particular model [6].

Important qualification

Using Fourier transforms does not automatically impose the heat equation. The FNO is trained from reference fields. Conservation, positivity, and boundary behavior must be imposed or checked separately.

Chapter 8

The Actual Prototype Architecture

8.1 Input normalization

The network input has two channels:

$$a_1(x) = \frac{\log k(x) - \mu_{\log k}}{s_{\log k}}, \qquad a_2(x) = 2x - 1. \tag{8.1}$$

The archived training statistics are

$$\mu_{\log k} = -0.000818, \qquad s_{\log k} = 0.627006. \tag{8.2}$$

Only the training set is used to compute them.

The target is scaled by the maximum training temperature:

$$\widetilde{T}(x) = \frac{T(x)}{T_{\text{scale}}}, \qquad T_{\text{scale}} = 0.344317. \tag{8.3}$$

8.2 Lifting, residual blocks, and projection

The lifting layer maps two inputs to 32 latent channels:

$$v_0 = P[a_1, a_2]. \tag{8.4}$$

The code uses four residual Fourier blocks. The actual update is

$$\boxed{v_{\ell+1} = v_\ell + 0.5\ \text{GELU}\left(\mathcal{K}_\ell v_\ell + W_\ell v_\ell\right), \qquad \ell = 0, 1, 2, 3.} \tag{8.5}$$

This is more specific than the generic poster equation because it includes the residual connection and factor 0.5 used in the script.

The projection is

$$32 \xrightarrow{1\times1} 64 \xrightarrow{\text{GELU}} 1. \tag{8.6}$$

Finally, the two boundary outputs are overwritten:

$$T_\theta(0) = T_\theta(1) = 0. \tag{8.7}$$

8.3 Why the boundary error is exactly zero

Equation (8.7) is an assignment operation, not a learned relation. Consequently, the maximum boundary error in the archived predictions is exactly zero. This is good architectural practice for fixed Dirichlet boundaries, but it must not be interpreted as evidence that the network learned boundary physics.

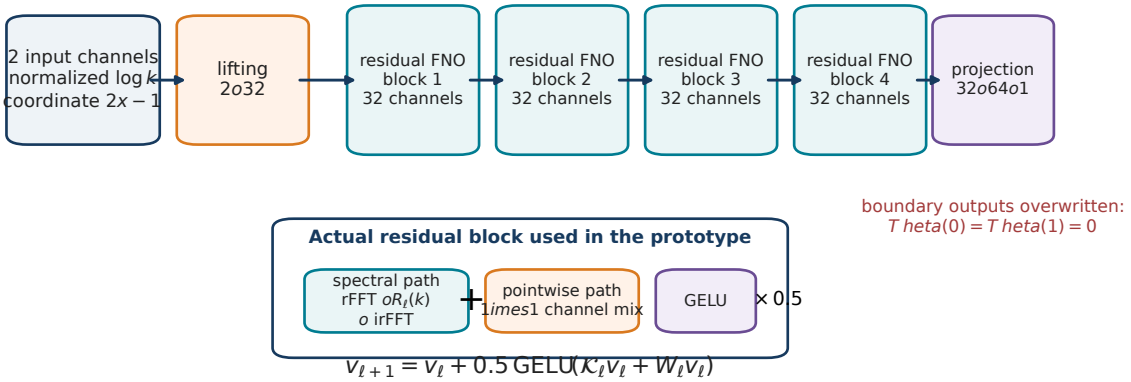


Figure 8.1: Exact architecture implemented in the supplied training script. The boundary condition is hard-coded after projection.

8.4 Spectral tensor shape

Each block has a complex spectral weight tensor

$$R_\ell \in \mathbb{C}^{32\times 32\times 16}. \tag{8.8}$$

Hence each block contains

$$32 \times 32 \times 16 = 16{,}384 \tag{8.9}$$

complex tensor elements, and four blocks contain

$$4 \times 16{,}384 = 65{,}536 \tag{8.10}$$

complex elements.

8.5 Detailed parameter count

The parameter count is derived in table 8.1.

Table 8.1: Parameter-count derivation for the actual prototype.

Component	Shape/count	Tensor elements	Real scalar DOF
Lifting weights and bias	$32 \times 2 + 32$	96	96
Four complex spectral tensors	$4(32 \times 32 \times 16)$	65,536	131,072
Four pointwise convolutions	$4(32 \times 32 + 32)$	4,224	4,224
First projection convolution	$64 \times 32 + 64$	2,112	2,112
Final projection convolution	$1 \times 64 + 1$	65	65
Total		**72,033**	**137,569**

PyTorch's `numel()` counts one complex number as one tensor element. A complex64 number contains two trainable real scalars. Thus both of the following statements are correct if their conventions are stated:

$$\text{tensor-element count} = 72,033, \qquad \text{real scalar degrees of freedom} = 137,569. \tag{8.11}$$

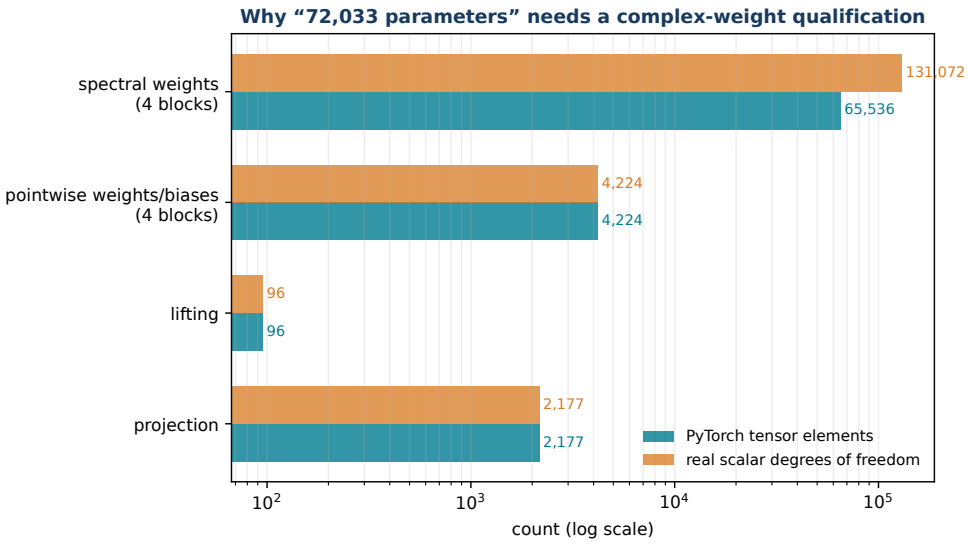


Figure 8.2: Breakdown of tensor elements and real scalar degrees of freedom. Complex spectral weights dominate the model capacity.

Check your understanding

Recalculate the total if the number of retained modes is reduced from 16 to 8, with all other dimensions unchanged. State both the tensor-element count and the real scalar degrees of freedom.

Chapter 9

Training Objective and Optimization Protocol

9.1 Supervised field loss

The normalized state mean-square loss for one batch is

$$\mathcal{L}_{\text{state}} = \frac{1}{BN} \sum_{i=1}^{B} \sum_{j=0}^{N-1} \left(\widetilde{T}_{\theta,j}^{(i)} - \widetilde{T}_{j}^{(i)} \right)^2 . \tag{9.1}$$

9.2 First-difference penalty

The training script adds

$$\mathcal{L}_{\Delta T} = \frac{1}{B(N-1)} \sum_{i=1}^{B} \sum_{j=0}^{N-2} \left[(\widetilde{T}_{\theta,j+1}^{(i)} - \widetilde{T}_{\theta,j}^{(i)}) - (\widetilde{T}_{j+1}^{(i)} - \widetilde{T}_{j}^{(i)}) \right]^2 . \tag{9.2}$$

The total objective is

$$\boxed{\mathcal{L} = \mathcal{L}_{\text{state}} + 0.03 \mathcal{L}_{\Delta T}.} \tag{9.3}$$

The gradient term is grid dependent

The code penalizes first differences, not derivatives divided by Δx. Therefore the numerical weight 0.03 is tied to the $N = 64$ grid and target normalization. A resolution-independent Sobolev loss would need explicit spatial scaling.

9.3 Optimizer and scheduler

The training protocol is:

- optimizer: AdamW;
- initial learning rate: 1.5×10^{-3};
- weight decay: 10^{-6};
- batch size: 100;
- epochs: 70;
- cosine annealing to 10^{-5};
- validation metric: mean relative L^2 error over 150 validation cases;
- model selection: checkpoint with the smallest validation error.

For cosine annealing, the nominal learning rate at epoch e is

$$\eta_e = \eta_{\min} + \frac{1}{2}(\eta_0 - \eta_{\min})\left[1 + \cos\left(\frac{\pi e}{70}\right)\right]. \tag{9.4}$$

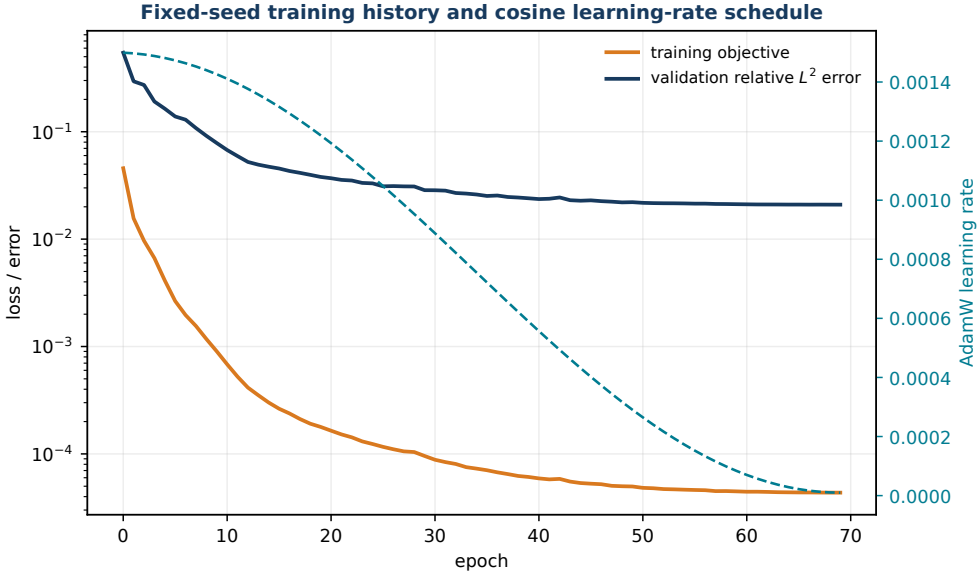


Figure 9.1: Archived fixed-seed training history and the cosine learning-rate schedule. The plotted training objective and validation relative error are different quantities and should not be compared as if they had identical definitions.

The best validation relative error is 2.0942%. The final epoch is also the best epoch in the archived run, but the code is written to restore the best checkpoint even if later epochs deteriorate.

9.4 Determinism and its limits

The fixed CPU environment reproduces the run to floating-point precision. This is computational reproducibility for a specified environment. It does not quantify:

- variability across random seeds;
- sensitivity to PyTorch or FFT-library versions;
- differences between CPU and GPU kernels;
- model-selection uncertainty;
- uncertainty in the data-generating distribution.

A publication-grade model comparison should repeat training over several seeds.

Part IV

Metrics, Results, and Physical Interpretation

Chapter 10

Derivation of the Evaluation Metrics

10.1 Temperature-field relative error

For each test case,

$$E_T = \frac{\|T_\theta - T\|_2}{\|T\|_2 + 10^{-12}}. \tag{10.1}$$

The Euclidean norm is taken over all 64 nodal values. Because both boundaries are zero in reference and prediction, they contribute neither numerator nor denominator.

The reported statistic is the median over a test family:

$$\widetilde{E}_T = \text{median}\{E_T^{(1)}, \ldots, E_T^{(250)}\}. \tag{10.2}$$

The median is robust to a few extreme cases but does not describe tail risk.

10.2 Heat-flux relative error

The face conductivity is computed from the input field:

$$k_{j+1/2} = \frac{2k_j k_{j+1}}{k_j + k_{j+1}}. \tag{10.3}$$

Reference and predicted face fluxes are

$$q_{j+1/2} = -k_{j+1/2}\frac{T_{j+1} - T_j}{\Delta x}, \tag{10.4}$$

$$q_{\theta,j+1/2} = -k_{j+1/2}\frac{T_{\theta,j+1} - T_{\theta,j}}{\Delta x}. \tag{10.5}$$

Then

$$E_q = \frac{\|\boldsymbol{q}_\theta - \boldsymbol{q}\|_2}{\|\boldsymbol{q}\|_2 + 10^{-12}}. \tag{10.6}$$

10.3 Why differentiation magnifies error

Let

$$T_\theta(x) = T(x) + \varepsilon \sin(2\pi m x). \tag{10.7}$$

The state perturbation amplitude is ε, but

$$\frac{\mathrm{d}}{\mathrm{d}x}(T_\theta - T) = 2\pi m\varepsilon \cos(2\pi m x). \tag{10.8}$$

The derivative error grows linearly with mode number m. A visually small oscillatory state error can therefore produce a much larger heat-flux error.

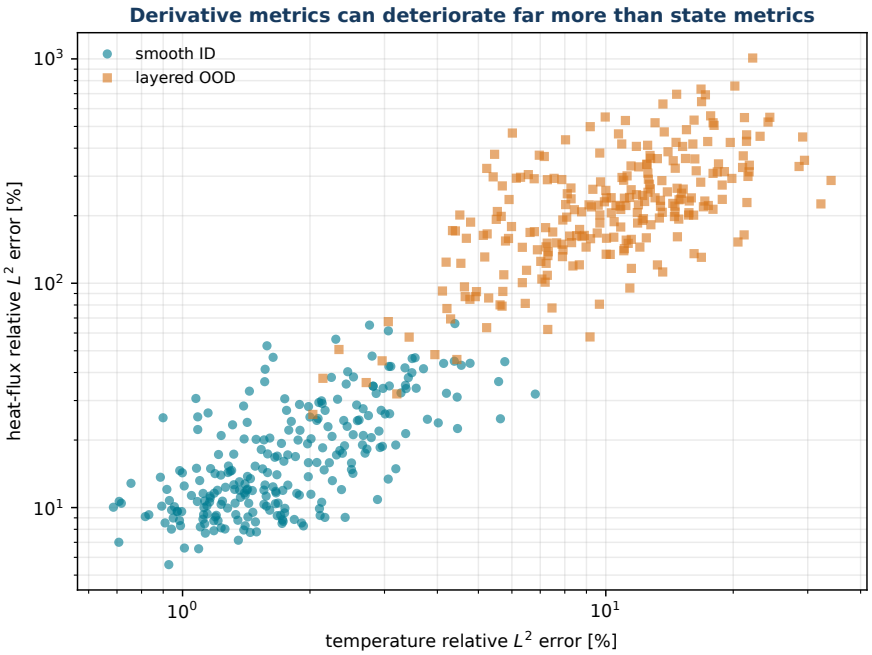


Figure 10.1: Case-by-case relationship between state and flux errors. Flux error is consistently larger and becomes extreme for layered OOD fields.

10.4 Discrete balance error

Using equation (4.14), define

$$E_{\text{bal}} = \frac{\left| q_{\theta,N-3/2} - q_{\theta,1/2} - (N-2)\Delta x \right|}{(N-2)\Delta x}. \tag{10.9}$$

The reference solution satisfies this algebraic balance to roundoff. A direct field predictor has no reason to satisfy it unless conservation is built into training or architecture.

10.5 Local PDE residual

For each internal stencil,

$$r_{\theta,j} = \frac{q_{\theta,j+1/2} - q_{\theta,j-1/2}}{\Delta x} - 1. \tag{10.10}$$

The per-case RMS residual is

$$r_{\text{RMS}} = \left[\frac{1}{N-2} \sum_{j=1}^{N-2} r_{\theta,j}^2 \right]^{1/2}. \tag{10.11}$$

This residual has the units of the nondimensional source and is not normalized by a reference residual because the reference residual is essentially zero.

10.6 Boundary and positivity diagnostics

Boundary error is

$$E_{\text{BC}} = \max\left(|T_\theta(0)|, |T_\theta(1)| \right). \tag{10.12}$$

It is exactly zero because of hard overwrite.

The positivity check records whether any internal prediction satisfies

$$T_{\theta,j} < -10^{-10}. \tag{10.13}$$

The threshold avoids classifying roundoff-level negative values as physical violations.

10.7 Bootstrap interval for the median

For a fixed array of 250 case errors, bootstrap resampling draws 250 values with replacement, computes the median, and repeats this process 20,000 times. The 2.5th and 97.5th percentiles form a nonparametric 95% interval [7].

This interval quantifies finite-test-sample uncertainty for the fixed trained model. It does not include training-seed uncertainty.

Audited Numerical Results

11.1 Four principal medians

Audited result

For the 250 unseen smooth ID fields,

$$\boxed{\text{median } E_T = 1.6670\%, \qquad \text{median } E_q = 14.6799\%.} \tag{11.1}$$

For the 250 layered OOD fields,

$$\boxed{\text{median } E_T = 10.8465\%, \qquad \text{median } E_q = 220.6089\%.} \tag{11.2}$$

All four values are independently recomputed from the archived fields and reproduced by a fresh fixed-seed run.

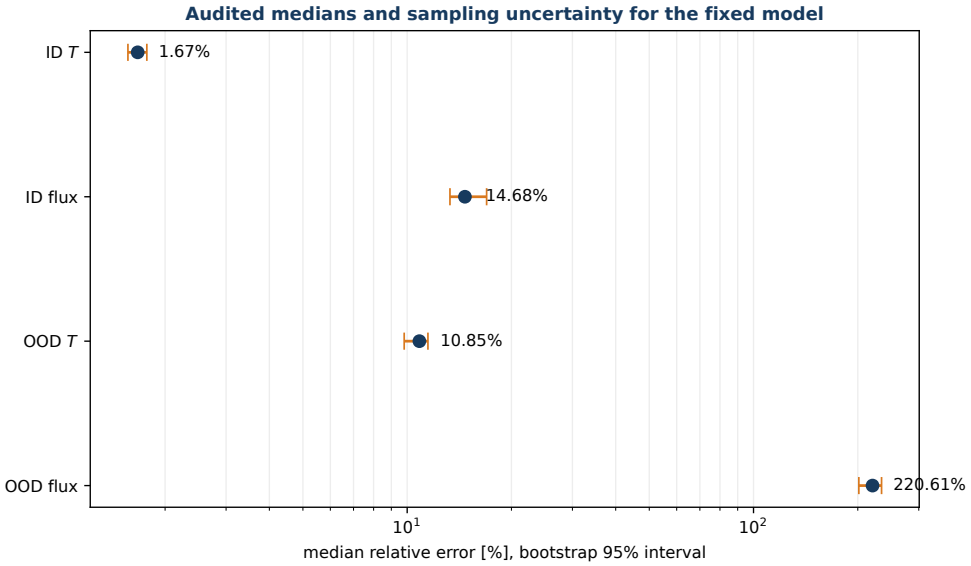


Figure 11.1: Audited medians with bootstrap intervals for the fixed trained model and fixed test families.

11.2 Full state-error distributions

The ID temperature distribution has

$$\text{mean } E_T^{\text{ID}} = 1.9629\%, \tag{11.3}$$
$$Q_{0.05} = 0.9292\%, \tag{11.4}$$
$$Q_{0.25} = 1.2359\%, \tag{11.5}$$
$$Q_{0.75} = 2.4321\%, \tag{11.6}$$
$$Q_{0.95} = 3.9158\%, \tag{11.7}$$
$$\max E_T^{\text{ID}} = 6.8130\%. \tag{11.8}$$

The 95% bootstrap interval for the median is $[1.5622\%, 1.7714\%]$.

For layered OOD temperature,

$$\text{mean } E_T^{\text{OOD}} = 11.4359\%, \tag{11.9}$$
$$Q_{0.05} = 4.3184\%, \tag{11.10}$$
$$Q_{0.25} = 7.2121\%, \tag{11.11}$$
$$Q_{0.75} = 14.6905\%, \tag{11.12}$$
$$Q_{0.95} = 21.5335\%, \tag{11.13}$$
$$\max E_T^{\text{OOD}} = 34.0617\%, \tag{11.14}$$

with median interval $[9.8041\%, 11.4798\%]$.

11.3 Full flux-error distributions

For ID heat flux,

$$\text{median } E_q = 14.6799\%, \qquad Q_{0.95} = 43.5460\%, \qquad \max E_q = 66.0862\%. \tag{11.15}$$

For layered OOD heat flux,

$$\text{median } E_q = 220.6089\%, \qquad Q_{0.95} = 527.6401\%, \qquad \max E_q = 1009.0461\%. \tag{11.16}$$

The OOD maximum exceeds 1000%, meaning the L^2 norm of the flux error is more than ten times the norm of the correct flux in the worst archived case.

11.4 Physical diagnostics

The median balance errors are

$$\text{median } E_{\text{bal}}^{\text{ID}} = 12.5234\%, \qquad \text{median } E_{\text{bal}}^{\text{OOD}} = 153.0085\%. \tag{11.17}$$

The median RMS residuals are

$$\text{median } r_{\text{RMS}}^{\text{ID}} = 3.1766, \qquad \text{median } r_{\text{RMS}}^{\text{OOD}} = 58.6715. \tag{11.18}$$

At least one negative internal temperature occurs in 5.6% of ID cases and 44.8% of OOD cases. The minimum predictions are -0.00309 and -0.02971, respectively.

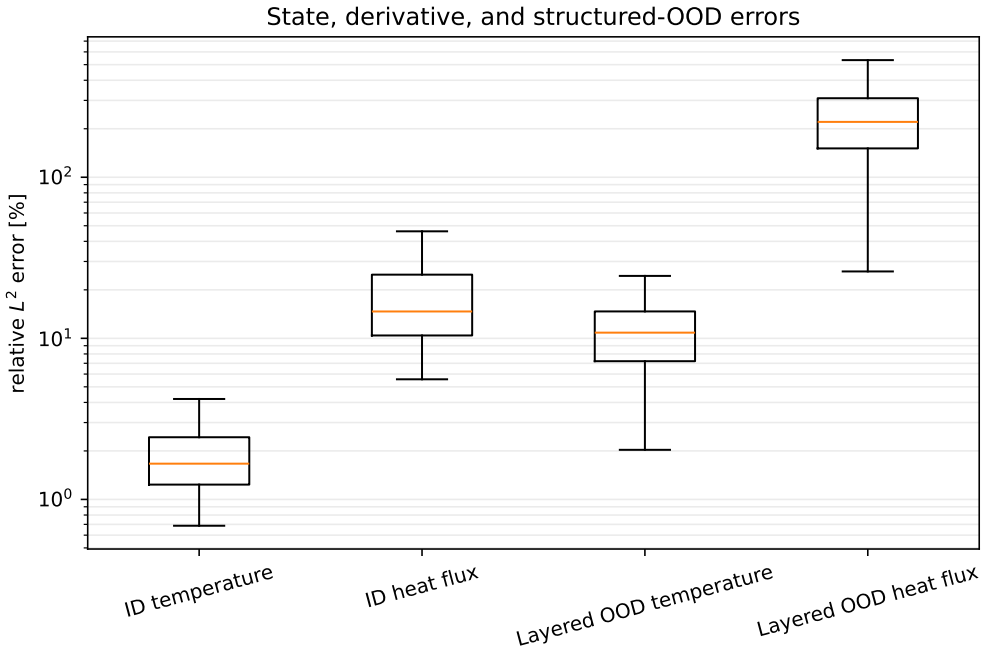


Figure 11.2: Error distributions over the two 250-case test families. The logarithmic scale is essential because OOD flux errors are very large.

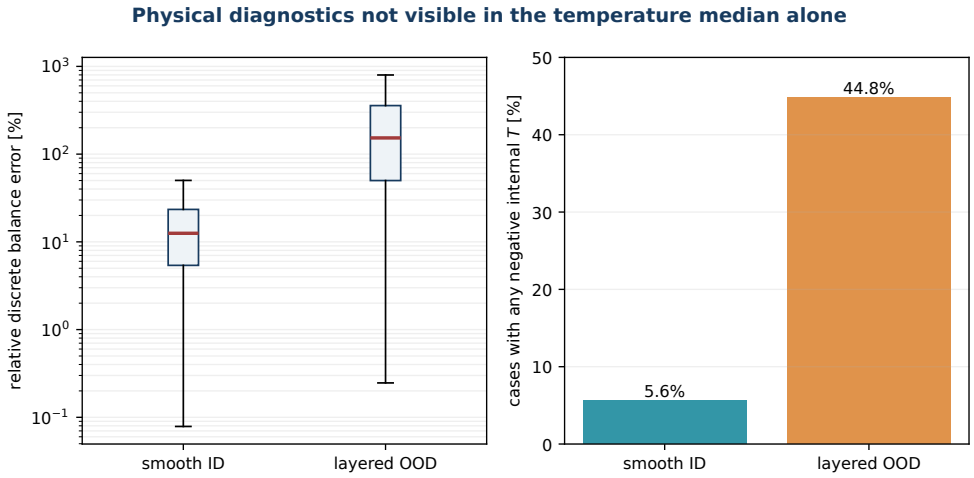


Figure 11.3: Balance and positivity diagnostics. These reveal physical failures not visible in the median temperature error.

Engineering interpretation

The ID state result is useful as an interpolation demonstration, but the flux and balance results show that the prototype is not conservative. The OOD results show that a smooth-field training distribution is insufficient for layered materials.

11.5 What the training history does and does not show

The training objective decreases steadily, and the validation temperature error reaches 2.0942%. This demonstrates successful optimization for the chosen supervised objective. It does not demonstrate:

- minimum heat-flux error;
- minimum balance error;
- robustness across seeds;
- superiority to a CNN baseline;
- resolution transfer;
- measured inference speedup;
- experimental validity.

Chapter 12

Representative Cases and Local Error Structure

12.1 How the representative cases are selected

The poster displays the case whose temperature error is closest to the family median. This is not the case with median flux error or median balance error. Specifically:

$$\text{ID representative index} = 154, \tag{12.1}$$

$$\text{OOD representative index} = 14. \tag{12.2}$$

For the ID representative,

$$E_T = 1.6632\%, \qquad E_q = 11.63\%, \qquad E_{\text{bal}} = 10.18\%. \tag{12.3}$$

For the OOD representative,

$$E_T = 10.8475\%, \qquad E_q = 213.24\%, \qquad E_{\text{bal}} = 200.47\%. \tag{12.4}$$

12.2 Why the ID temperature profile can look excellent

The temperature solution is an integral of the flux divided by conductivity. Integration smooths local errors. Consequently, a predicted temperature can match the broad parabolic-like profile while containing small local oscillations or slope errors. Those slope errors are exposed by the heat-flux calculation.

12.3 Why layered fields are difficult

Several mechanisms act together:

1. Layer interfaces are absent from the training distribution.
2. A discontinuity requires broad Fourier content.
3. The spectral path retains only 16 of 33 nonredundant modes.
4. The output temperature remains smooth, but its derivative must change sharply enough to preserve flux across conductivity jumps.
5. The training loss places only a small weight on unscaled first differences and no direct weight on physical flux or balance.

At a material interface, continuous flux requires

$$k^{-}T'^{-} = k^{+}T'^{+}. \tag{12.5}$$

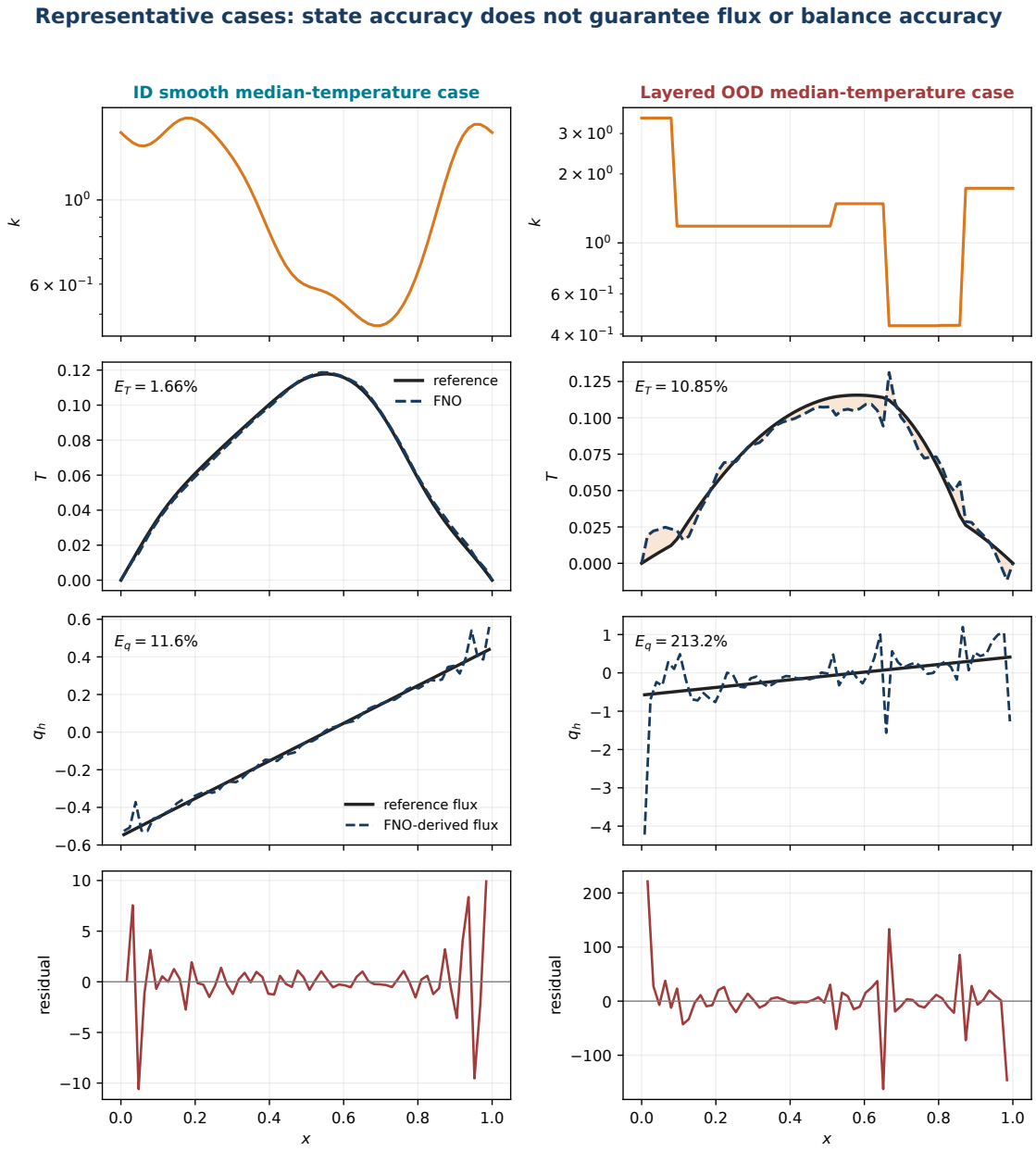


Figure 12.1: Detailed diagnostics for the median-temperature representative cases. The ID temperature curves nearly overlap, but its flux and residual are visibly less accurate. The layered case shows severe flux and residual errors, including a negative temperature region.

A conductivity jump therefore produces a reciprocal jump in temperature gradient. Capturing the temperature field visually is easier than capturing this interface condition accurately.

12.4 A spectral interpretation with an important caveat

The layered inputs have broader spectra than the smooth inputs. This supports the interpretation that spectral truncation contributes to difficulty. However, the experiment does not isolate architecture from data coverage. The model has never been trained on layers, so one cannot conclude that adding modes alone would solve the problem. A controlled ablation would be required.

Important qualification

The note distinguishes an evidence-supported explanation from a demonstrated causal result. The archived experiment establishes a correlation between held-out layered structure and larger error. It does not separately quantify the effects of mode count, local path, loss function, or training distribution.

Chapter 13

Independent Reproduction and Audit Trail

13.1 Archived checkpoint prediction check

The saved checkpoint loads directly into the supplied `FNO1d` class. In the original audited environment, re-evaluation reproduced the archived prediction arrays exactly. A second verification in the packaged environment produced maximum absolute differences of 1.04×10^{-7} for ID predictions and 1.19×10^{-7} for layered-OOD predictions, consistent with float32 roundoff rather than a scientifically meaningful change.

13.2 Independent metric recomputation

The verification script ignores stored error arrays when recomputing the metrics. It reads k, T, and T_θ, reconstructs face fluxes using equations (4.3) and (4.8), and evaluates equations (10.1) and (10.6). The recomputed arrays agree exactly with the stored arrays.

13.3 Independent retraining

A fresh run regenerates the same data and retrains from seed 20260805. The four medians are reproduced to better than 0.0001 percentage points. The logged training time in the audited CPU environment is approximately 20.6 seconds, but this is an environment-specific observation and is not used as a poster speedup claim.

13.4 File integrity

The audit records SHA-256 hashes for:

- `prototype_results.npz`;
- `fno_prototype_model.pt`;
- `train_heat_fno_prototype.py`;
- `generate_fno_poster_assets.py`.

Hashes make it possible to detect accidental changes to evidence files.

13.5 Reproducibility hierarchy

It is useful to separate four levels:

1. **Traceability:** every poster number can be located in code or data.
2. **Recomputation:** metrics can be recalculated from archived fields.
3. **Re-execution:** the checkpoint regenerates archived predictions.

4. **Retraining:** data generation and model optimization reproduce the result from the fixed seed.

The package passes all four for the stated environment. A multi-seed robustness study remains future work.

Part V

Scientific Interpretation and Next Steps

Chapter 14

What the Results Support – and What They Do Not

14.1 Supported statement

Scientifically supportable conclusion

In a fixed-seed synthetic benchmark for one-dimensional heterogeneous heat conduction, a compact FNO achieved a median relative temperature error of 1.6670% on 250 unseen smooth conductivity fields. The corresponding median heat-flux error was 14.6799%. For 250 layered fields absent from training, the temperature and heat-flux medians increased to 10.8465% and 220.6089%. Independent recomputation and retraining reproduced these values. The study is a reproducible operator-learning proof of concept, not experimental or field-scale validation.

14.2 Statements not supported by the experiment

The following claims should not be made from this prototype:

- the model is validated for subsurface storage, CCUS, pipelines, or thermal systems;
- the model is conservative;
- a finer output grid recovers missing physics;
- the model is robust across random seeds;
- the FNO is superior to CNN, U-Net, DeepONet, or reduced-order baselines;
- the method provides a measured speedup factor;
- the model is reliable for discontinuous conductivity fields;
- the model can be used safely for inverse design or control.

14.3 Why median temperature error alone is insufficient

A median summarizes a typical case, not the tail. A state norm summarizes the field, not its derivative or balance. For engineering decisions, one may care about:

- maximum temperature;
- boundary heat rate;
- local heat flux;
- interface continuity;
- energy balance;

- positivity;
- threshold exceedance;
- sensitivity to material changes.

A surrogate can be visually convincing and physically inadequate.

14.4 Model card for the prototype

Table 14.1: Compact model card for the audited prototype.

Intended use	Undergraduate demonstration of field-to-field operator learning and validation methodology.
Governing problem	Steady 1D heterogeneous heat conduction with unit source and fixed zero Dirichlet boundaries.
Training distribution	Smooth positive log-Fourier conductivity fields.
Primary test	Unseen smooth fields from the same stochastic family.
Structured OOD test	Piecewise-constant layered fields absent from training.
Strength	Low median ID temperature-field error.
Weaknesses	Larger flux error, nonconservation, occasional negative temperatures, severe layered-OOD flux failure.
Not validated for	Experiments, field systems, irregular geometry, transients, multiphase flow, inverse problems, or safety-critical use.

Chapter 15

How to Improve the Prototype Scientifically

15.1 Multi-seed evaluation

Repeat the complete experiment for at least five, preferably ten, independent seeds. For each metric, report both:

- variability across test cases for each trained model;
- variability across training runs.

A hierarchical summary prevents a lucky seed from being mistaken for robust performance.

15.2 Comparable baselines

Train a one-dimensional CNN or U-Net-like model with a comparable real scalar parameter budget. Compare:

$$E_T, \qquad E_q, \qquad E_{\mathrm{bal}}, \qquad \text{parameter count}, \qquad \text{training cost}, \qquad \text{inference cost}. \tag{15.1}$$

The point is not to declare a universal winner, but to identify which architecture handles global coupling and interfaces more effectively.

15.3 Flux-aware and residual-aware loss

A physically aligned objective could be

$$\mathcal{L} = \lambda_T \mathcal{L}_T + \lambda_q \mathcal{L}_q + \lambda_r \mathcal{L}_r + \lambda_{\mathrm{pos}} \mathcal{L}_{\mathrm{pos}}. \tag{15.2}$$

Here

$$\mathcal{L}_q = \frac{1}{B} \sum_i \frac{\left\| \boldsymbol{q}_\theta^{(i)} - \boldsymbol{q}^{(i)} \right\|_2^2}{\left\| \boldsymbol{q}^{(i)} \right\|_2^2 + \varepsilon}, \tag{15.3}$$

$$\mathcal{L}_r = \frac{1}{B(N-2)} \sum_{i,j} r_{\theta,j}^{(i)2}, \tag{15.4}$$

$$\mathcal{L}_{\mathrm{pos}} = \frac{1}{B(N-2)} \sum_{i,j} \max(0, -T_{\theta,j}^{(i)})^2. \tag{15.5}$$

Loss weights must be tuned on validation metrics, not on final tests.

15.4 Conservative output design

Instead of predicting T directly, a network could predict face flux corrections:

$$q_{\theta,j+1/2} = q_{\mathrm{coarse},j+1/2} + \Delta q_{\theta,j+1/2}. \tag{15.6}$$

A conservative reconstruction would enforce

$$\frac{q_{\theta,j+1/2} - q_{\theta,j-1/2}}{\Delta x} = 1 \tag{15.7}$$

or solve a small correction system. This design aligns the learned representation with the conservation law.

15.5 Mixed smooth-layered training

Train on controlled mixtures of smooth and layered fields. Use held-out combinations of layer count, jump magnitude, and interface location. This separates missing training physics from insufficient spectral capacity.

15.6 True resolution-transfer experiment

A defensible test would:

1. generate the same continuous conductivity realization;
2. solve it independently on $N = 64$, 128, and 256 grids;
3. train at $N = 64$;
4. evaluate the learned operator at finer grids with consistent normalization;
5. compare with genuine fine-grid reference solutions;
6. evaluate state, flux, balance, and interface errors.

Producing a smooth field on a finer grid is not sufficient evidence of super-resolution.

Chapter 16

Pathway to Porous-Media and Energy Applications

16.1 Single-phase Darcy analogue

The one-dimensional heat equation and steady Darcy pressure equation have similar elliptic structure. For isotropic single-phase flow,

$$-\nabla \cdot \left[\frac{K(\boldsymbol{x})}{\mu} \nabla p(\boldsymbol{x}) \right] = q(\boldsymbol{x}). \tag{16.1}$$

A basic operator is

$$[\log K(\boldsymbol{x}), q(\boldsymbol{x}), \text{BCs}] \mapsto p(\boldsymbol{x}). \tag{16.2}$$

Pressure has global coupling, making Fourier layers attractive.

16.2 Why the heat prototype is only a first step

Subsurface applications introduce challenges absent from the prototype. Geometry-aware variants, including learned-deformation approaches, illustrate one route beyond rectangular grids [8]:

- two- and three-dimensional geometry;
- anisotropic tensor permeability;
- wells and near-singular pressure gradients;
- inactive cells, faults, and irregular boundaries;
- compressibility and time dependence;
- multiphase saturation fronts;
- component conservation and phase behavior;
- operational metrics such as bottom-hole pressure and breakthrough time.

16.3 Global-local conservative operator

A promising research architecture combines:

$$\text{global Fourier pressure branch} + \text{local front-resolution branch} + \text{conservative flux update}. \tag{16.3}$$

Pressure and saturation should not necessarily share identical decoders. Pressure is often smooth and globally coupled, whereas saturation contains sharp moving interfaces.

16.4 Validation ladder for future energy models

Before deployment, a porous-media neural operator should pass:

1. numerical solver verification;
2. interpolation tests on unseen geological realizations;
3. flux and component-mass balance checks;
4. held-out facies, correlation lengths, wells, and controls;
5. long-horizon stability tests;
6. comparison with baseline surrogates;
7. gradient verification for inversion or optimization;
8. decision-level tests for pressure limits, plume extent, and breakthrough.

Key idea

The main research direction is not simply to apply a larger FNO. It is to build neural operators whose architecture, targets, and validation metrics reflect the conservation laws and multiscale structure of the engineering system.

Part VI

Student Exercises and Reproducibility Materials

Chapter 17

Guided Exercises

17.1 Analytical exercises

1. For $k(x) = k_0 \exp(\alpha x)$, evaluate $R(x)$ and $A(x)$ analytically and write $T(x)$ using equation (3.10).
2. Generalize the exact flux construction to a source $f(x) = 1 + x$.
3. Prove that $C \in [-1, 0]$ for positive $k(x)$.
4. Show that the point $x_* = -C$ is where the heat flux changes sign and the temperature reaches its maximum.
5. For a two-layer conductor with conductivities k_1, k_2 and interface a, derive the piecewise temperature and enforce continuity of temperature and heat flux.

17.2 Numerical exercises

1. Assemble the tridiagonal matrix for $N = 6$ explicitly.
2. Verify the discrete global balance equation (4.15) by summing all internal equations.
3. Compare arithmetic and harmonic face conductivity for a jump $k_j = 1$, $k_{j+1} = 100$. Which better represents series resistance?
4. Reproduce the smooth solver convergence table and calculate the observed order with equation (5.2).
5. Modify the nodal discretization to use boundary half-control-volumes and derive the new source inventory.

17.3 Machine-learning exercises

1. Reduce retained modes from 16 to 8 and derive the new parameter count before training.
2. Replace the unscaled first-difference loss with a derivative loss divided by Δx. Explain how its numerical magnitude changes.
3. Train on a 50–50 mixture of smooth and layered fields. Compare ID, layered, and mixed held-out metrics.
4. Add a flux loss and examine whether state error increases while flux error decreases.
5. Train five random seeds and separate within-test-set variability from between-training-run variability.
6. Compare hard boundary overwrite with a boundary penalty.
7. Predict both T and q_h as outputs, then add a consistency loss enforcing $q_h = -kT'$.

17.4 Discussion questions

1. Is a 1.67% median temperature error “accurate”? What application information is needed before answering?
2. Why can a model with exact boundary values still violate global conservation?
3. Does the layered OOD failure primarily diagnose architecture, data distribution, or both?
4. What acceptance criteria would you set for a thermal safety application?
5. Which results would be required before extending the conclusion to two-dimensional Darcy flow?

Chapter A

Key Code Excerpts

The complete scripts are supplied in the `code/` directory. This appendix highlights the exact operations that generate the poster results.

A.1 Reference solver

```
np.random.seed(SEED)
random.seed(SEED)
torch.manual_seed(SEED)
torch.set_num_threads(max(1, min(8, os.cpu_count() or 1)))

x = np.linspace(0.0, 1.0, N_GRID)
dx = x[1] - x[0]


def solve_heat(k: np.ndarray) -> np.ndarray:
    """Solve the heterogeneous steady heat equation with a Thomas algorithm."""
    n_internal = N_GRID - 2
    k_face = 2.0 * k[:-1] * k[1:] / (k[:-1] + k[1:] + 1.0e-30)

    lower = -k_face[1:n_internal] / dx**2
    upper = lower.copy()
    diagonal = (k_face[:n_internal] + k_face[1 : n_internal + 1]) / dx**2
    rhs = np.ones(n_internal)

    # Forward elimination.
    for j in range(1, n_internal):
        multiplier = lower[j - 1] / diagonal[j - 1]
        diagonal[j] -= multiplier * upper[j - 1]
        rhs[j] -= multiplier * rhs[j - 1]

    # Back substitution.
    internal = np.empty(n_internal)
    internal[-1] = rhs[-1] / diagonal[-1]
    for j in range(n_internal - 2, -1, -1):
        internal[j] = (rhs[j] - upper[j] * internal[j + 1]) / diagonal[j]
```

Listing A.1: Flux-form heat solver with harmonic face conductivity and Thomas algorithm.

A.2 Conductivity generators

```
    temperature[1:-1] = internal
    return temperature


def smooth_conductivity() -> np.ndarray:
    """Smooth positive log-Fourier random field used for training and ID tests."""
    mean_log_k = np.random.uniform(-0.3, 0.3)
    log_k = np.full(N_GRID, mean_log_k)
    amplitude = np.random.uniform(0.4, 0.8)
```

```
    for mode in range(1, 6):
        scale = amplitude / mode**1.7
        log_k += scale * (
            np.random.randn() * np.sin(2.0 * np.pi * mode * x)
            + np.random.randn() * np.cos(2.0 * np.pi * mode * x)
        )

    # Limit the strongest contrasts while retaining a broad smooth family.
    standard_deviation = log_k.std()
    if standard_deviation > 0.9:
        log_k = (
            (log_k - log_k.mean()) * 0.9 / standard_deviation + log_k.mean()
        )
    return np.exp(log_k)


def layered_conductivity() -> np.ndarray:
    """Piecewise-constant OOD field family never used during training."""
    n_layers = np.random.randint(3, 7)
    cuts = sorted(
        np.random.choice(np.arange(4, N_GRID - 4), n_layers - 1, replace=False)
    )
    indices = [0, *cuts, N_GRID]
    values = np.exp(
        np.random.uniform(np.log(0.25), np.log(4.0), size=n_layers)
    )
    field = np.empty(N_GRID)
    for layer in range(n_layers):
        field[indices[layer] : indices[layer + 1]] = values[layer]
    return field


def generate_dataset(n_samples: int, family: str = "smooth") -> tuple[np.ndarray, np.ndarray]:
    generator = smooth_conductivity if family == "smooth" else layered_conductivity
    conductivity = np.stack([generator() for _ in range(n_samples)]).astype(np.float32)
    temperature = np.stack([solve_heat(k) for k in conductivity]).astype(np.float32)
```

Listing A.2: Smooth training family and layered OOD family.

A.3 Spectral layer and residual block

```

class SpectralConv1d(nn.Module):
    """Low-mode complex spectral convolution."""

    def __init__(self, in_channels: int, out_channels: int, n_modes: int) -> None:
        super().__init__()
        self.n_modes = n_modes
        scale = 1.0 / (in_channels * out_channels)
        self.weight = nn.Parameter(
            scale
            * torch.randn(
                in_channels, out_channels, n_modes, dtype=torch.cfloat
            )
        )

    def forward(self, values: torch.Tensor) -> torch.Tensor:
        spectrum = torch.fft.rfft(values, dim=-1)
```

```
        output_spectrum = torch.zeros(
            values.shape[0],
            self.weight.shape[1],
            spectrum.shape[-1],
            dtype=torch.cfloat,
            device=values.device,
        )
        n_active = min(self.n_modes, spectrum.shape[-1])
        output_spectrum[:, :, :n_active] = torch.einsum(
            "bim,iom->bom",
            spectrum[:, :, :n_active],
            self.weight[:, :, :n_active],
        )
        return torch.fft.irfft(output_spectrum, n=values.shape[-1], dim=-1)


class FNOBlock1d(nn.Module):
    def __init__(self, width: int, n_modes: int) -> None:
        super().__init__()
        self.spectral = SpectralConv1d(width, width, n_modes)
        self.pointwise = nn.Conv1d(width, width, kernel_size=1)
        self.activation = nn.GELU()

    def forward(self, values: torch.Tensor) -> torch.Tensor:
        update = self.activation(self.spectral(values) + self.pointwise(values))
        return values + 0.5 * update


class FNO1d(nn.Module):
    def __init__(self, width: int = 32, n_modes: int = 16, n_layers: int = 4) -> None:
        super().__init__()
        self.lifting = nn.Conv1d(2, width, kernel_size=1)
        self.blocks = nn.ModuleList(
            [FNOBlock1d(width, n_modes) for _ in range(n_layers)]
        )
        self.projection = nn.Sequential(
            nn.Conv1d(width, 64, kernel_size=1),
            nn.GELU(),
            nn.Conv1d(64, 1, kernel_size=1),
        )

    def forward(self, inputs: torch.Tensor) -> torch.Tensor:
        latent = self.lifting(inputs)
        for block in self.blocks:
            latent = block(latent)
        output = self.projection(latent).clone()
        # Exact projection of the two fixed Dirichlet boundary values.
        output[:, :, 0] = 0.0
        output[:, :, -1] = 0.0
        return output


def relative_l2(prediction: torch.Tensor, target: torch.Tensor) -> torch.Tensor:
    numerator = torch.linalg.vector_norm(prediction - target, dim=(-2, -1))
    denominator = torch.linalg.vector_norm(target, dim=(-2, -1)).clamp_min(1.0e-6)
    return (numerator / denominator).mean()
```

Listing A.3: Complex low-mode spectral convolution, residual FNO block, projection, and hard boundary overwrite.

A.4 Training and metric calculation

```
    model = FNO1d()
    print("Trainable parameters:", sum(p.numel() for p in model.parameters()))

    train_loader = DataLoader(
        TensorDataset(
            torch.from_numpy(make_input(k_train)),
            torch.from_numpy(make_target(t_train)),
        ),
        batch_size=100,
        shuffle=True,
    )
    validation_input = torch.from_numpy(make_input(k_validation))
    validation_target = torch.from_numpy(make_target(t_validation))

    optimizer = torch.optim.AdamW(model.parameters(), lr=1.5e-3, weight_decay=1.0e-6)
    scheduler = torch.optim.lr_scheduler.CosineAnnealingLR(
        optimizer, N_EPOCHS, eta_min=1.0e-5
    )

    history: list[tuple[float, float]] = []
    best_validation = float("inf")
    best_state: dict[str, torch.Tensor] | None = None
    start = time.time()

    for epoch in range(N_EPOCHS):
        model.train()
        batch_losses = []
        for inputs, targets in train_loader:
            optimizer.zero_grad()
            prediction = model(inputs)
            state_loss = ((prediction - targets) ** 2).mean()
            gradient_loss = (
                (torch.diff(prediction, dim=-1) - torch.diff(targets, dim=-1)) ** 2
            ).mean()
            loss = state_loss + 0.03 * gradient_loss
            loss.backward()
            optimizer.step()
            batch_losses.append(float(loss.detach()))

        scheduler.step()
        model.eval()
        with torch.no_grad():
            validation_error = float(
                relative_l2(model(validation_input), validation_target)
            )
        history.append((float(np.mean(batch_losses)), validation_error))

        if validation_error < best_validation:
            best_validation = validation_error
            best_state = {
                name: parameter.detach().clone()
                for name, parameter in model.state_dict().items()
            }
        if epoch % 10 == 0:
            print(epoch, history[-1], "best", best_validation)

    if best_state is None:
        raise RuntimeError("Training did not produce a model checkpoint.")
```

```
    model.load_state_dict(best_state)
    print(f"Training time: {time.time() - start:.2f} s")

    def predict(k: np.ndarray) -> np.ndarray:
        model.eval()
        with torch.no_grad():
            normalized = model(torch.from_numpy(make_input(k))).numpy()[:, 0]
        return normalized * temperature_scale

    def metrics(k: np.ndarray, t: np.ndarray) -> tuple[np.ndarray, np.ndarray, np.ndarray]:
        prediction = predict(k)
        state_error = np.linalg.norm(prediction - t, axis=1) / (
            np.linalg.norm(t, axis=1) + 1.0e-12
        )
        k_face = 2.0 * k[:, :-1] * k[:, 1:] / (k[:, :-1] + k[:, 1:] + 1.0e-30)
        true_flux = -k_face * np.diff(t, axis=1) / dx
        predicted_flux = -k_face * np.diff(prediction, axis=1) / dx
        flux_error = np.linalg.norm(predicted_flux - true_flux, axis=1) / (
            np.linalg.norm(true_flux, axis=1) + 1.0e-12
        )
        return prediction, state_error, flux_error

    prediction_test, error_test, flux_test = metrics(k_test, t_test)
    prediction_ood, error_ood, flux_ood = metrics(k_ood, t_ood)

    print("ID median temperature error:", np.median(error_test))
    print("ID median heat-flux error:", np.median(flux_test))
    print("Layered-OOD median temperature error:", np.median(error_ood))
    print("Layered-OOD median heat-flux error:", np.median(flux_ood))

    np.savez(
        ROOT / "prototype_results.npz",
        x=x,
        ktr=k_train,
        ttr=t_train,
        kte=k_test,
        tte=t_test,
```

Listing A.4: Normalization, training objective, checkpoint selection, and field/flux metrics.

Chapter B

Complete Audited Statistics

Table B.1: Audited relative-error statistics. All values are percentages except where noted.

Metric	median	mean	5th pct.	25th pct.	75th pct.	95th pct.	maximum	95% CI median
ID temperature	1.6670	1.9629	0.9292	1.2359	2.4321	3.9158	6.8130	1.5622–1.7714
ID heat flux	14.6799	19.2221	8.1562	10.4177	24.8483	43.5460	66.0862	13.2907–16.9587
OOD temperature	10.8465	11.4359	4.3184	7.2121	14.6905	21.5335	34.0617	9.8041–11.4798
OOD heat flux	220.6089	247.3280	68.2892	151.0286	309.4131	527.6401	1009.0461	201.4084–234.2454

B.1 Additional physical diagnostics

Table B.2: Physical diagnostics of the direct temperature predictor.

Metric	Smooth ID	Layered OOD
Median relative discrete balance error	12.5234‰	153.0085‰
Median RMS residual	3.1766	58.6715
Cases with any negative internal temperature	5.6%	44.8%
Minimum predicted temperature	-0.00309	-0.02971
Maximum boundary error	0	0

Chapter C

Selected Solutions to Exercises

C.1 Bound on the integration constant

Because $1/k(x) > 0$, the ratio

$$-C = \frac{\int_0^1 x/k(x)\,\mathrm{d}x}{\int_0^1 1/k(x)\,\mathrm{d}x} \tag{C.1}$$

is a weighted average of $x \in [0, 1]$. Hence $-C \in [0, 1]$ and $C \in [-1, 0]$.

C.2 Location of the temperature maximum

From equation (3.4), $T'(x) = 0$ when $x = -C$. Since $-C \in [0, 1]$, this point lies in the domain. The flux changes sign there, so heat generated on either side flows toward the corresponding boundary. Positivity and the boundary values imply this stationary point is the maximum.

C.3 Parameter count with eight modes

With eight modes, the four spectral tensors contain

$$4 \times 32 \times 32 \times 8 = 32{,}768 \tag{C.2}$$

complex tensor elements. The real-valued components remain 6,497. Therefore

$$\text{tensor elements} = 32{,}768 + 6{,}497 = 39{,}265, \tag{C.3}$$

$$\text{real scalar DOF} = 2(32{,}768) + 6{,}497 = 72{,}033. \tag{C.4}$$

Interestingly, the new real scalar count equals the original model's PyTorch tensor-element count; the conventions must not be confused.

Chapter D

Reproduction Guide

From the source directory:

```
python code/train_heat_fno_prototype.py
python code/verify_fno_prototype.py
python code/generate_derivation_note_figures.py
pdflatex FNO_Poster_Results_Detailed_Derivation_Note.tex
bibtex FNO_Poster_Results_Detailed_Derivation_Note
pdflatex FNO_Poster_Results_Detailed_Derivation_Note.tex
pdflatex FNO_Poster_Results_Detailed_Derivation_Note.tex
```

The archived results and model are in `data/`. The machine-readable audit and independent retraining log are in `audit/`.

Re-running versus preserving evidence

Running the training script overwrites the archived result and checkpoint files in its own directory. Preserve a copy of the verified package before experimenting with code changes.

Chapter E

Notation

Symbol	Meaning
$k(x)$	thermal conductivity field
$T(x)$	reference temperature field
$T_\theta(x)$	FNO-predicted temperature field
$q_h(x)$	conductive heat flux, $-kT'$
$\mathcal{G}^\dagger$	reference PDE solution operator
$\mathcal{G}_\theta$	learned neural operator
$\mathcal{F}, \mathcal{F}^{-1}$	Fourier and inverse Fourier transforms
$R_\ell(k)$	complex spectral channel-mixing matrix in layer ℓ
W_ℓ	pointwise channel-mixing operator
N	number of nodal grid points, 64 in the prototype
M	number of retained real-FFT modes, 16 in the prototype
E_T	relative temperature-field L^2 error
E_q	relative face-flux L^2 error
E_{bal}	relative discrete global-balance error
r_{RMS}	RMS local discrete PDE residual
ID	in-distribution smooth-field test family
OOD	structured out-of-distribution layered-field test family

Bibliography

[1] Nikola Kovachki, Zongyi Li, Burigede Liu, Kamyar Azizzadenesheli, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Neural operator: Learning maps between function spaces with applications to pdes. *Journal of Machine Learning Research*, 24(89):1–97, 2023.

[2] Suhas V. Patankar. *Numerical Heat Transfer and Fluid Flow*. Hemisphere Publishing Corporation, 1980.

[3] Randall J. LeVeque. *Finite Difference Methods for Ordinary and Partial Differential Equations: Steady-State and Time-Dependent Problems*. Society for Industrial and Applied Mathematics, 2007.

[4] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations*, 2021.

[5] Lloyd N. Trefethen. *Spectral Methods in MATLAB*. Society for Industrial and Applied Mathematics, 2000.

[6] Nikola Kovachki, Samuel Lanthaler, and Siddhartha Mishra. On universal approximation and error bounds for fourier neural operators. *Journal of Machine Learning Research*, 22(290):1–76, 2021.

[7] Bradley Efron and Robert J. Tibshirani. *An Introduction to the Bootstrap*. Chapman and Hall/CRC, 1993.

[8] Zongyi Li, Daniel Zhengyu Huang, Burigede Liu, and Anima Anandkumar. Fourier neural operator with learned deformations for pdes on general geometries. *Journal of Machine Learning Research*, 24(388):1–26, 2023.